



Anwendungshandbuch für die Software SILAB® PROFESSIONAL

Version 7.1

Mail: silab@wivw.de
Telefon: +49 931 78009-400



INHALTSVERZEICHNIS

1	VORWORT.....	4
2	EINFÜHRUNG UND ÜBERBLICK.....	5
2.1	Übersicht über SILAB.....	5
2.2	Ziel und Adressaten des Handbuchs.....	6
2.3	Inhalte des Handbuchs	6
2.4	Code-Erläuterungen in diesem Handbuch	7
3	BESTANDTEILE EINER KARTE	9
4	AUFBAU DER KONFIGURATIONSDATEI	11
4.1	Aufbau der Szenario-Definition	11
5	GESTALTUNG VON STRECKENSTÜCKEN	15
5.1	Gestaltung von Courses.....	15
5.1.1	Querschnittsprofil.....	16
5.1.2	Lageplan.....	21
5.1.3	Höhenprofil des Straßenverlaufs.....	23
5.1.4	Umgebungslandschaft	24
5.1.5	Grafische Objekte	28
5.1.6	Strecken-Events	29
5.2	Gestaltung von Areas	32
5.2.1	Einbinden von Areas in die Konfigurationsdatei.....	33
5.3	Nachbau realer Strecken	36
6	ERZEUGUNG EINES STRECKENNETZES AUS STRECKENSTÜCKEN	37
7	VERKEHRSSIMULATION FÜR EIN STRECKENNETZ	40
7.1	Aufbau der Verkehrsdefinition in der Konfigurationsdatei.....	40
7.1.1	Allgemeine Angaben zur Verkehrssimulation.....	40
7.1.2	Verkehrsdefinition für ein Streckennetz.....	41
7.2	Verhaltensmodelle und Verhaltensschemata	42
7.3	Verhaltenssteuerung durch Flowpoints	42
7.4	Definition einzelner Verkehrsteilnehmer	43
7.5	Definition eines Verkehrsflusses	49

7.6	Fußgängersimulation	54
8	ERZEUGUNG EINER KARTE AUS SZENARIO-MODULEN.....	55
8.1	Festlegen der Aufsetzpunkte	58
9	WIEDERVERWENDUNG VON SZENARIO-MODULEN	61
10	ERWEITERUNG DER GRAFISCHEN AUSGABE	62
11	GLOSSAR	63

1 VORWORT

Wir freuen uns, Ihnen mit SILAB die passende Software für Ihren Fahrsimulator zu liefern. SILAB ist die von der WIVW GmbH entwickelte Fahrsimulationssoftware, die sowohl für PKW- als auch für LKW-, Motorrad- oder Fahrradsimulatoren verwendet werden kann.

SILAB wird seit über 15 Jahren kontinuierlich im engen Austausch mit Partnern und Kunden aus Industrie und Forschung weiterentwickelt und ist für die unterschiedlichsten Anwendungsbereiche optimiert. Der Einsatz der Software eignet sich für die Forschung und Entwicklung von Verkehrs- und Fahrzeugsystemen, für diagnostische Fragestellungen (z.B. Beurteilung der Fahrleistung), für Präsentationszwecke (z.B. Veranschaulichungen von neuartigen Konzepten) sowie für Fahrtraining und Rehabilitation.

2 EINFÜHRUNG UND ÜBERBLICK

2.1 Übersicht über SILAB

SILAB ist eine Fahrsimulationssoftware, mit der Fahrsimulatoren auf sehr unterschiedlichen Ausbaustufen betrieben werden können. Die Software bietet die Möglichkeit, das Verhalten des simulierten Fahrzeugs und anderer Verkehrsteilnehmer, die virtuelle Umgebung, sowie Fahrzeug- und Umgebungsgeräusche realistisch und flexibel zu simulieren. Zur Streckengestaltung können bereits erprobte und sofort einsetzbare Szenarien mithilfe einer bedienfreundlichen grafischen Benutzeroberfläche zusammengesetzt werden, deren Anwendung keinerlei Programmierkenntnisse erfordert. Darüber hinaus können Anwender selber Szenarien entwerfen, die sie flexibel wiederverwenden und modifizieren können. Das Streckennetz von SILAB nutzt eine dynamische Datenbasis, die große Vorteile bei der Planung und Durchführung von Simulationsfahrten gegenüber anderer Fahrsimulationssoftware bietet. Im Gegensatz zu einem Streckennetz in der Realität kann mit SILAB das Streckennetz und alle dazugehörigen Variablen (andere Verkehrsteilnehmer, Landschaft usw.) außerhalb des Sichtbarkeitsbereichs des Fahrers beliebig verändert werden. SILAB ermöglicht das Überwachen diverser Parameter während der Simulation, sowie die Aufzeichnung von Messgrößen und Videos. Es können externe Hardware- und Softwarekomponenten über flexible Schnittstellen in die Simulation eingebunden werden.

SILAB ist in drei verschiedenen Editionen erhältlich, welche aufeinander aufbauen und unterschiedliche Nutzeranforderungen bedienen.

Die Edition ESSENTIAL ist für Anwender geeignet, die Untersuchungen oder Trainings mit bereits bestehenden Szenarien durchführen möchten. Auf Basis umfangreicher Datenbanken aus Stadt-, Autobahn-, und Landstraßen-Szenarien können diese verschiedenen Szenarien einfach zu individuellen Strecken zusammengesetzt werden. Es bestehen die Möglichkeiten, externe Hard- und Software (Bedienelemente, Blickverhalten, Kommunikationsschnittstellen usw.) einzubinden, sowie Messwerte, Audio- und Videodaten aus der Simulation zeitsynchron aufzuzeichnen.

Die Edition PROFESSIONAL richtet sich an Anwender, die eigene Szenarien oder komplexe Straßennetze schnell und effizient gestalten möchten. Hierzu stehen ein grafischer Editor sowie eine einfache Skriptsprache zur Verfügung.

Die Edition ENTERPRISE ist für Anwender konzipiert, die eigene Software für die Simulation entwickeln und integrieren möchten. Dafür stehen Schnittstellen für die Programmiersprachen C/C++, Java und Ruby zur Verfügung. Die Schnittstellen erlauben Zugriff auf die Streckengeometrie, die Beschilderung, die Verkehrsteilnehmer (Egofahrer und simulierte Verkehrsteilnehmer) und deren Eigenschaften/Zustände sowie nahezu alle Werte, die in SILAB verarbeitet und aufgezeichnet werden können. Hiermit ist es möglich, Assistenzsysteme, Sensorsysteme, Nebenaufgaben, komplexe Human-Machine-Interfaces (HMIs) usw. in SILAB zu integrieren.

2.2 Ziel und Adressaten des Handbuchs

Dieses Handbuch vermittelt Kenntnisse zum Gestalten eigener Szenarien sowie zum Erstellen ganzer Streckennetzwerke bzw. Karten. Es demonstriert Ihnen die vielfältigen Möglichkeiten von SILAB zur Gestaltung der Simulation nach Ihren eigenen Wünschen. Das Handbuch richtet sich an Benutzer der SILAB-Editionen PROFESSIONAL und ENTERPRISE.

Wir möchten Ihnen mit diesem Handbuch das Wissen zum Arbeiten mit SILAB so vermitteln, dass Sie auch ohne Informatikkenntnisse lernen, selbst Szenarien zu erstellen und zu Karten zu kombinieren. Zum Erzielen schneller Erfolgserlebnisse bei der Konfiguration von Szenarien sind die Erklärungen dieses Handbuchs kurzgehalten und werden anhand von anschaulichen Beispielen erklärt.

Das Ziel besteht darin, ein übergeordnetes Verständnis für die Gestaltung von Szenarien und ganzen Versuchsabläufen in der Konfigurationsdatei zu vermitteln. Das Handbuch hat nicht den Anspruch, alle Details zu den verwendeten Programmen, DPUs und zur Skriptsprache zu erläutern, sondern verweist stattdessen an mehreren Stellen auf Dokumente mit weiterführenden Informationen.

Ausgangspunkt für die Nutzung dieses Handbuchs ist die bereits geschehene Inbetriebnahme Ihres Simulators. Außerdem baut dieses Handbuch auf Kenntnissen, die im Handbuch SILAB Essential vermittelt werden, auf. Wir empfehlen, dass Sie zum Einstieg in das vorliegende Handbuch bereits über Kenntnisse zu folgenden Inhalten verfügen:

- Verwaltung der Simulationsrechner mit dem Programm SILABAdmin
- Zusammenstellung von Sessions mit dem Programm SILABSessionEdit
- Steuerung der Simulation und Überwachung von Parametern während der Simulation mit dem Programm SILAB
- Wissen über DPUs und deren Aufgaben in SILAB
- Aufbau von Konfigurationsdateien
- Aufbau und Bearbeiten der System-, Rechner- und DPU-Konfiguration

2.3 Inhalte des Handbuchs

Das Handbuch erklärt, wie Sie mithilfe der Skriptsprache in der Konfigurationsdatei die einzelnen Bestandteile eines Szenarios definieren können und die verschiedenen Szenario-Module zu einer gesamten Karte zusammensetzen. Auf das Programm SILABAEEdit, das zur Gestaltung von Streckenstücken mit komplexeren Spurgeometrien eingesetzt wird, wird an mehreren Stellen verwiesen. Die Anleitung zur Nutzung des Programms finden Sie jedoch in einem gesonderten Dokument (Handbuch SILABAEEdit).

In Kapitel 3 werden zunächst Begriffserklärungen zu den verschiedenen Bestandteilen, aus denen eine Karte in SILAB besteht, eingeführt. Es folgt eine Übersicht über den Aufbau der Szenario-Definition in der Konfigurationsdatei (Kapitel 4). Daraufhin lernen Sie Schritt für Schritt, wie Sie die einzelnen Komponenten eines Course-Streckenstücks in der Konfigurationsdatei definieren und wie Sie Area-Streckenstücke in die Konfigurationsdatei einbinden (Kapitel 5). Area-Streckenstücke werden mit dem grafischen Editor SILABAEEdit erstellt und erst im Anschluss zu der übergeordneten Konfiguration hinzugefügt. In Kapitel 6 erfahren Sie, wie Sie verschiedene Area- und Course-Streckenstücke miteinander zu einem Streckennetz verknüpfen. Kapitel 7 beschreibt das Vorgehen zur Definition des Verkehrs für ein Streckennetz. Daraufhin wird in Kapitel 8 auf die Erzeugung einer Karte aus Szenario-Modulen eingegangen. In Kapitel 9 erfahren Sie, wie Sie bereits erstellte Szenario-Module für andere Projekte wiederverwenden können und in Kapitel 10, wie Sie die grafische Ausgabe von SILAB erweitern können.

An verschiedenen Stellen des Handbuchs wird auf **WEITERFÜHRENDE DOKUMENTE** verwiesen, mit denen Sie Ihr Wissen je nach Ihren Interessen und Aufgabenstellungen erweitern bzw. vertiefen können. Die entsprechenden Dokumente befinden sich in dem Ordner D:\SILAB\doc.

Das Handbuch enthält diverse Kästen mit Informationen und Warnhinweisen. Info- und Hinweis-Kästen sehen folgendermaßen aus:

Informationen und Hinweise werden mit Hilfe eines solchen Kastens hervorgehoben.

2.4 Code-Erläuterungen in diesem Handbuch

In diesem Handbuch werden an verschiedenen Stellen Codes eingefügt, die Beispiele für bestimmte Konfigurationen der Szenariodefinition enthalten. Diese Codes erkennen Sie an der Benennung ‚Code‘ mit fortlaufender Nummerierung. Regeln zur syntaktisch korrekten Anwendung der Skriptsprache zur Bearbeitung der Konfigurationsdateien finden Sie in Kapitel 8.4.2 im **HANDBUCH SILAB ESSENTIAL**.

In den eingefügten Codes wird der Beginn eines Abschnitts immer mit dem Kommentar ‚### Abschnittsname‘ und das Ende mit ‚### /Abschnittsname‘ markiert. Die Ordnung der Unterabschnitte erkennen Sie außerdem daran, dass die Abschnitte unterschiedlich stark eingerückt sind.

Die Codes selbst enthalten Beispielkonfigurationen. Es wird innerhalb der Codes bewusst auf Platzhalter verzichtet, damit Sie eine bessere Übersicht über die Erstellung der Anweisungen in der Konfigurationsdatei erhalten. Im Anschluss an die Codes folgen Anmerkungen zu den Regeln bzw. den formalen Grammatiken, nach denen diese Konfigurationen erstellt werden. In diesen Erläuterungen wird von Ersetzungsregeln in der Form von <...> Gebrauch gemacht. Diese Bezeichner in spitzen Klammern sind sogenannte Nichtterminalsymbole. Das bedeutet, dass ein solches Symbol niemals selbst Bestandteil des Codes ist, sondern im Code nach bestimmten Regeln ersetzt wird. Bei der Anweisung

LandscapeTypeLeft <Landschaftstypname>

Wird <Landschaftstypname> im eigentlichen Code bspw. durch einen von 24 vordefinierten Landschaftstypen ersetzt. z.B.:

LandscapeTypeLeft Farmed

Zu jedem Bezeichner in spitzen Klammern werden Erklärungen gegeben und es wird angegeben, durch welche Möglichkeiten sie im Code ersetzt werden können.

Des Weiteren gibt es teilweise alternative Möglichkeiten für die Ersetzung von Anweisungen. Dies wird durch das Zeichen | kenntlich gemacht, z.B.:

Position = (<Streckenstück-Instanz>, <Streckenmeter>, <Spurnummer>);|

Position = (SimCar, <Abstand>, <Spurnummer>);

3 BESTANDTEILE EINER KARTE

In diesem Kapitel lernen Sie, aus welchen Bestandteilen eine sogenannte Karte in SILAB besteht und wie diese Bestandteile definiert werden. Als Karte (engl. Map) wird in SILAB ein gesamtes Streckennetzwerk bezeichnet, auf dem man in der Simulation fahren kann. Zur Erstellung eigener Karten in der Konfigurationsdatei von SILAB werden die einzelnen Bestandteile einer Karte zunächst definiert und daraufhin zu einer gesamten Karte zusammengesetzt. Der Aufbau und die Bestandteile einer Karte werden in Abbildung 1 veranschaulicht.

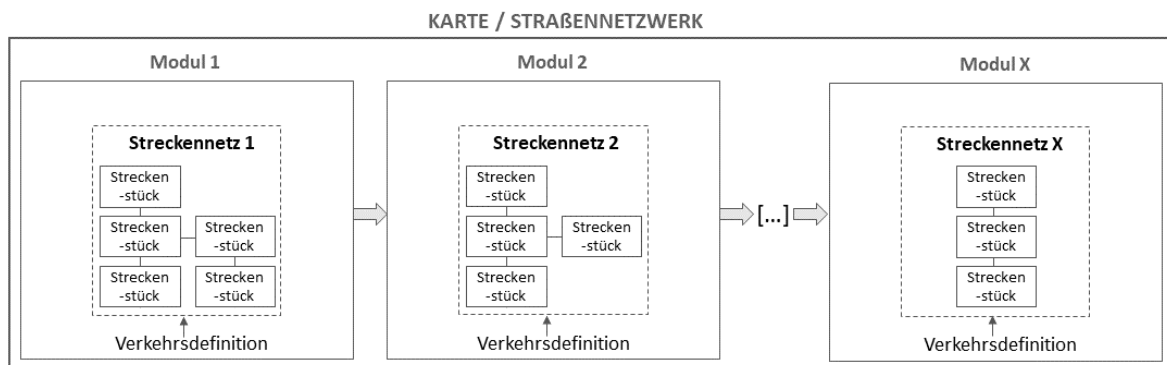


Abbildung 1. Bestandteile einer Karte.

Der kleinste Bestandteil einer Karte ist ein **STRECKENSTÜCK**. Streckenstücke können bspw. ein durchgängiger Abschnitt einer Straße, eine Kreuzung, eine Autobahnauffahrt oder -abfahrt sein. Auf den Streckenstücken können sogenannte **STRECKEN-EVENTS** platziert werden. Dabei handelt es sich um Punkte, die für den Fahrer unsichtbar sind und Ereignisse in der Simulation auslösen können, z.B. das Auslösen einer Navigationsansage.

Aus der Verknüpfung mehrerer Streckenstücke wird ein **STRECKENNETZ** erzeugt.

Für ein solches Streckennetz erfolgt eine **VERKEHRSDEFINITION**. Sie beinhaltet Vorschriften, wie sich andere einzelne Verkehrsteilnehmer oder ganze Verkehrsflüsse auf dem vorgegebenen Streckennetz verhalten sollen. Die **FUßGÄNGERDEFINITION** gibt vor, wie sich die Fußgänger auf diesem Streckennetz verhalten sollen.

Als **SZENARIO-MODUL** wird eine abgeschlossene Einheit aus einem Streckennetz und einer Verkehrsdefinition bezeichnet. Diese Szenario-Module werden zu einer bestimmten Reihenfolge verknüpft, in der sie dem Fahrer letztlich während der Simulation dargeboten werden (Modul-Abfolge).

Das gesamte Straßennetzwerk, das eine Einheit aus den genannten Bestandteilen bildet, wird als **KARTE** bezeichnet.

Ein weiterer wichtiger Begriff für die Erstellung eigener Karten ist die **DATENBASIS**. Als Datenbasis wird die in der Simulation vorhandene Umgebung bezeichnet. Die Datenbasis enthält Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes sowie das

Aussehen und das Höhenprofil der Umgebung. Streckennetze in SILAB werden mit der Datenbasis namens SCN/SCNX erstellt.

Eine Session, die mit dem Programm SILABSessionEdit erstellt wurde und als Konfigurationsdatei exportiert wurde, wird *Parcours* genannt. Aus technischer Sicht kann ein solcher *Parcours* ebenfalls als Karte bezeichnet werden. Der Unterschied zwischen einem *Parcours* und einer Karte ist, dass im *Parcours* eine eindeutige Abfolge der Szenarien vorgegeben ist (durch die Herkunft aus SILABSessionEdit). Wenn man die Karte selbst konfiguriert, kann die Abfolge der Szenarien-Module flexibler gestaltet werden.

4 AUFBAU DER KONFIGURATIONSDATEI

In diesem Abschnitt erhalten Sie einen Überblick über den Aufbau der Szenariodefinition in der Konfigurationsdatei. Konfigurationsdateien sind Textdateien, die eine Art Bauplan für die Gestaltung der Simulation beinhalten. Sie bestehen aus unterschiedlichen Abschnitten, in denen die Bestandteile der Simulation definiert werden:

- Systemkonfiguration: Konfiguration grundlegender Einstellungen des SILAB-Systems
- Rechnerkonfiguration: Definition der Rechner, die an der Simulation beteiligt sind
- DPU-Konfiguration: Definition der Softwaremodule, die an der Simulation beteiligt sind
- Allgemeine Angaben zu weiteren Komponenten der Simulation
- Allgemeine Angaben zur Verkehrssimulation
- Szenariodefinition: Definition von Strecke, Umgebung, Verhalten von Verkehrsteilnehmern und Fußgängern für verschiedene Szenarien

Eine Einführung zum Arbeiten mit Konfigurationsdateien sowie Möglichkeiten zur Modifizierung der System-, Rechner- und DPU-Konfiguration finden Sie im **HANDBUCH SILAB ESSENTIAL** (Kapitel 8). Dort finden Sie auch einen Überblick über die include-Dateien, die in der Standardkonfiguration enthalten sind und üblicherweise als Basis für jede Konfigurationsdatei dienen.

Die Szenariodefinition ist der Abschnitt der Konfigurationsdatei, in dem die Bestandteile einer Karte definiert und verknüpft werden. Das Straßennetzwerk und das Verhalten der anderen Verkehrsteilnehmer sind meist sehr spezifisch und müssen daher für jede Fragestellung, die mit der Simulation untersucht wird, angepasst werden. Allerdings gibt es auch bei der Szenario-Definition die Möglichkeit, an bestimmten Stellen sogenannte include-Dateien zu erstellen und einzufügen, in denen bspw. wiederverwendbare Querschnittsprofile von Straßen definiert werden (s. Abschnitt 5.1.1).

4.1 Aufbau der Szenario-Definition

Code 1 (auf der nächsten Seite) zeigt den Aufbau der Szenario-Definition in der Konfigurationsdatei. Die Szenario-Definition ist in Unterabschnitte unterschiedlicher Ordnung aufgeteilt. Sie wird in der Konfigurationsdatei mit `SILAB SCN { . . . }` eingeleitet.

Zu Beginn der Szenario-Definition wird die Grundkonfiguration der SILAB-Datenbasis SCNX als include-Datei in die Konfigurationsdatei eingebunden (`include scnx/SCNX_700.cfg`). Die Datenbasis enthält sowohl Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes als auch über Aussehen und Höhenprofil der Umgebung.

Die Szenariodefinition besteht aus der Definition unterschiedlicher Szenariomodule, die letztlich zu einer Karte zusammengefügt werden. In dem hier gezeigten Beispiel würden zwei Szenariomodule X und Y definiert.

Die Definition eines Szenariomoduls bildet die erste Unterebene der Szenariodefinition in der Konfigurationsdatei. Die Definition eines Moduls wird immer mit der Anweisung `define Module` eingeleitet, gefolgt von einem eindeutigen Namen für das Modul (im Beispiel werden die Namen `ModulX` und `ModulY` vergeben). Jedem Modul wird eine eindeutige Kennung (`ModuleID`) zugewiesen. Die Werte der `ModuleID` stehen als Ausgänge der DPUSCNX zur Verfügung (`ModuleTypeID` und `ModuleInstanceID`) und können in eine Spalte der Aufzeichnungsdatei geschrieben werden. In Kombination mit der ID des dazugehörigen Streckenabschnitts können Abschnitte des Versuchs identifiziert werden und die aufgezeichneten Daten im Nachhinein zugeordnet werden. Sie können eine beliebige Ziffer als `ModuleID` auswählen.

Die Definition eines Szenario-Moduls besteht aus drei Unterebenen, auf deren Definitionen in den folgenden Kapiteln genauer eingegangen wird:

- Definition der Streckenstücke: Courses und Areas (Kapitel 5)
- Erzeugung eines Streckennetzes aus den Streckenstücken (Kapitel 6)
- Verkehrsdefinition für das Streckennetz (Kapitel 7)

Auf der gleichen Unterebene wie die Definitionen der Szenariomodule befindet sich der Abschnitt, in dem die Modulabfolge der Szenariomodule festgelegt wird (Erzeugung der Karte aus den Szenariomodulen). In diesem Abschnitt werden die Module zu einer Karte zusammengefügt.

```
### Szenariodefinition
SILAB SCN
{
### Grundkonfiguration für die Szenariodefinition
include scn/SCNX_700.cfg
### /Grundkonfiguration für die Szenariodefinition

    ### Szenariomodul X
define Module ModulX
{
ModuleID = 0;

    ### Definition der Streckenstücke

### Courses
### /Courses

### Einfügen der Areas
### /Einfügen der Areas

    ### /Definition der Streckenstücke

    ### Erzeugung des Streckennetzes
    ### /Erzeugung des Streckennetzes

    ### Verkehrsdefinition
    ### /Verkehrsdefinition
};
    ### /Szenariomodul X

    ### Szenariomodul Y
define Module ModulY
{
ModuleID = 1;

    ### Definition der Streckenstücke

### Courses
### /Courses

### Einfügen der Areas
### /Einfügen der Areas

    ### /Definition der Streckenstücke

    ### Erzeugung des Streckennetzes
    ### /Erzeugung des Streckennetzes

    ### Verkehrsdefinition
    ### /Verkehrsdefinition
};
    ### /Szenariomodul Y
```

```
    ### Erzeugung der Karte aus den Szenariomodulen  
    ### /Erzeugung der Karte aus den Szenariomodulen  
}  
### /Szenariodefinition
```

Code 1. Aufbau der Szenario-Definition.

5 GESTALTUNG VON STRECKENSTÜCKEN

In diesem Kapitel erfahren Sie, welche Typen von Streckenstücken es in SILAB gibt und wie Sie diese Streckenstücke in der Konfigurationsdatei und mithilfe des Programms SILABAEEdit gestalten können. Zur Gestaltung von Streckenstücken gehört die Definition des Verlaufs und des Höhenprofils der Straße, sowie des Querschnittsprofils, die Ausgestaltung der umgebenen Landschaft und bei Bedarf die Einbindung bestimmter Strecken-Events.

In SILAB werden zwei Arten von Streckenstücken unterschieden. Ein **COURSE** ist ein beliebig langer Abschnitt einer Straße, der keine Abzweigungen hat und bei dem sich das Querschnittsprofil (z.B. Anzahl der Spuren, Spurbreiten, Belag der Spuren) nicht ändert. Streckenstücke, bei denen sich das Querschnittsprofil ändert, werden Area genannt. **AREAS** sind z.B.:

- eine Kreuzung,
- eine Autobahnauffahrt/-abfahrt,
- eine Spurverbreiterung/-verengung,
- ein Stück Straße, in dem Spuren dazu kommen oder wegfallen,
- ein Teil einer Ortschaft mit mehreren Kreuzungen und Verbindungsstraßen.

Areas haben dementsprechend eine deutlich komplexere Spurgeometrie als Courses. Während die Bestandteile eines Course in der Konfigurationsdatei definiert werden, wäre es sehr aufwändig, die komplexen Spurverläufe einer Area ebenso in der Konfigurationsdatei zu definieren. Deswegen enthält der Lieferumfang von SILAB ein Programm namens SILABAEEdit, welches als grafischer Editor zur Gestaltung von Areas genutzt wird. In Abschnitt 5.1 werden Sie schrittweise durch die Gestaltung von Courses mithilfe der Skriptsprache in der Konfigurationsdatei geführt. In Abschnitt 5.2 erhalten Sie einen kurzen Überblick über die Möglichkeiten des Programms SILABAEEdit zur Gestaltung von Areas und erfahren, wie Sie damit erstellte Area-Streckenstücke in der Konfigurationsdatei in das gewünschte Streckennetz integrieren können.

5.1 Gestaltung von Courses

Im Folgenden lernen Sie Schritt für Schritt, wie Sie die verschiedenen Bestandteile eines Course in der Konfigurationsdatei nach Ihren Wünschen konfigurieren. Die Bestandteile bestehen aus dem Querschnittsprofil der Straße, dem Lageplan, dem Höhenprofil des Straßenverlaufs, der Gestaltung der Umgebungslandschaft und bei Bedarf aus Strecken-Events. Ein Abschnitt in der Konfigurationsdatei, in dem die Bestandteile eines Course definiert werden, ist einem Modulabschnitt untergeordnet (s. Abschnitt 4). Code 2 zeigt den üblichen Aufbau einer Course-Definition in der Konfigurationsdatei mit seinen verschiedenen Unterabschnitten. Die Unterabschnitte werden im Lauf dieses Kapitels mit Inhalt befüllt.

```
### Courses
define Course Start      # Benennung des 1. Course
{
  NodeID = 0;            # Zuweisung einer eindeutigen ID für diesen
                          # Course

  ### Querschnittsprofil
  ### /Querschnittsprofil

  ### Lageplan
  ### /Lageplan

  ### Höhenprofil
  ### /Höhenprofil

  ### Landschaft
  ### /Landschaft

  ### Grafische Objekte
  ### /Grafische Objekte

  ### Strecken-Events
  ### /Strecken-Events

};
### /Courses
```

Code 2. Aufbau einer Course-Definition.

Die Definition eines Course wird immer mit der Anweisung `define Course` eingeleitet, gefolgt von einem eindeutigen Namen für den Course (im Beispiel wird der Name `Start` vergeben).

Jedem Streckenstück wird eine ID (`NodeID`) zugewiesen. Dabei handelt es sich um eine eindeutige Kennung für diesen Streckenabschnitt. In Kombination mit der ID des dazugehörigen Moduls können Abschnitte des Versuchs identifiziert werden. Die Werte der `NodeID` stehen als Ausgänge der `DPUSCNX` zur Verfügung (`NodeTypeID` und `NodeInstanceID`) und können somit in der Aufzeichnungsdatei in eine Spalte geschrieben werden. Dies ermöglicht im Nachhinein die Zuordnung der aufgezeichneten Daten zu einem Streckenabschnitt. Sie können eine beliebige Ziffer als `NodeID` auswählen. Das übliche Vorgehen ist das Hochzählen von 1 an für die verschiedenen Streckenstücke, die in der Konfigurationsdatei definiert werden.

5.1.1 Querschnittsprofil

Das Querschnittsprofil definiert den Aufbau einer Straße im Querschnitt und enthält folgende Informationen:

- Anzahl der Spuren
- Breite und Höhe der Spuren

- Fahrtrichtung und Befahrbarkeit
- Straßenbelag
- Linienmarkierungen zwischen den Spuren
- Bebauungen zwischen bzw. neben den Spuren (z.B. Leuchtpfosten, Leitplanken)
- Bebauungen auf den Spuren (z.B. Hecke auf einem Begrenzungstreifen in der Mitte einer Autobahn)
- Seitenstreifen: Übergang zwischen der äußersten Spur und der Landschaft

Querschnittsprofile von SILAB bieten sehr viele Gestaltungsmöglichkeiten. Sie haben entweder die Möglichkeit, diese Informationen einzeln für Ihren Course zu definieren oder auf ein bereits bestehendes Querschnittsprofil zurückzugreifen. Sie können sowohl Querschnittsprofile nutzen, die Sie in der Vergangenheit für einen anderen Course erstellt haben als auch solche, die im Lieferumfang von SILAB enthalten sind.

Im ersten Schritt werden die Anweisungen zur Definition eines Querschnittsprofils in der Konfigurationsdatei erläutert. Dies ist sowohl für die eigene Erstellung von Querschnittsprofilen, als auch zum Verständnis von bereits bestehenden Querschnitten hilfreich. Im zweiten Schritt erfahren Sie, wie Sie bereits bestehende Querschnittsprofile auf einen Course anwenden können.

Die Definition eines Querschnittsprofils in der Konfigurationsdatei sieht bspw. folgendermaßen aus:

```
### Querschnittsprofil

CrossSection Autobahnprofil
{
  ve = 150.0;
  Shoulder = {1.0, 2.4, -0.7, Shoulder1};
  Overlay = {TarNew1, 1.0};

  LaneInfos =
  {
    (0, 2.5, 0.0, Towards, Tar, 1.0, 0.0, {(None, 0.0)}),
    (1, 3.75, 0.0, Towards, Tar, 1.0, 0.0, {(None,
    0.0)}),
    (2, 3.75, 0.0, Towards, Tar, 1.0, 0.0, {(None,
    0.0)}),
    (3, 3.0, 0.0, Towards, Grass1, 1.0, 0.0, {(None,
    0.0)}),
    (4, 3.75, 0.0, Onwards, Tar, 1.0, 0.0, {(None,
    0.0)}),
    (5, 3.75, 0.0, Onwards, Tar, 1.0, 0.0, {(None,
    0.0)}),
    (6, 2.5, 0.0, Onwards, Tar, 1.0, 0.0, {(None, 0.0)})
  };

  LaneTransitions =
  {
    (0, 0,
```

Name für das
Quer-
schnittsprofil
Entwurfsge-
schw.
Standstreifen
Overlay-Textur

Infos über
Spuren, # 1
Zeile pro Spur
von links
nach # rechts

Übergänge
zwischen # Spuren

```

{(ReflectionPosts, -0.3, 1.0, Towards, 50.0, 0.0),      # linker
 (Line, 0.12, 0.15, Towards, 1.0, 0.0)}               # Fahrbahnrand
),
(1, 2,
 {(Line, 0.0, 0.15, Towards, 6.0, 12.0)}              # Übergang Spur
),                                                    1 zu # Spur 2
};
};

### /Querschnittsprofil

```

Code 3. Beispiel für die Definition eines Querschnittsprofils.

Im Folgenden werden der Aufbau und die Bestandteile der Querschnittsprofildefinition erläutert. Die spitzen Klammern dienen in den Erläuterungen nur als Platzhalter für die tatsächlichen Werte. Detailliertere Informationen zur Grammatik der Skriptsprache, mit der das Querschnittsprofil definiert wird, und zu den auswählbaren Parametereinstellungen (z.B. Straßenbelag oder Spurübergänge) finden Sie in Kapitel 2.5.5 im Dokument **REFERENZ DATENBASIS SCNX**.

<Name der Overlay- Textur>	Eine Übersicht über die Benennungen der verschiedenen Overlay-Texturen finden Sie in Kapitel 6.2 im Dokument REFERENZ DATENBASIS SCNX .
<Sichtbarkeit>	Die Sichtbarkeit des Overlays kann von 0 (unsichtbar) bis 1 (undurchsichtig) stufenlos gewählt werden, sodass auch eine Kombination mit der darunterliegenden Straßentextur möglich ist.

Im nächsten Abschnitt werden die einzelnen Spuren definiert. Jeder Eintrag in dieser Menge beschreibt eine Spur und enthält nacheinander folgende Informationen:

LaneInfos= (<Spurindex>,<Spurbreite in m>,<relative Höhe>, <Richtung>,<Straßenbelag>,<Reibbeiwert>,<Rauigkeit>, {(<Bebauungstyp>,<YOffset>)})	
<Spurindex>	Nummerierung der verschiedenen Spuren der Straße: Die Autobahn in Code 3 hat 7 Spuren, nämlich (von links nach rechts betrachtet): Standstreifen auf der Gegenfahrbahn (Spur 0), 2 Fahrspuren auf der Gegenfahrbahn (Spur 1 und 2), begrünter Mittelstreifen (Spur 3), 2 Fahrspuren in Fahrtrichtung (Spur 4 und 5), Standstreifen in Fahrtrichtung (Spur 6).
<relative Höhe>	Dieser Parameter wird derzeit nicht unterstützt und sollte stets als 0 angegeben werden.
<Richtung>	Entweder wird hier <code>Onwards</code> (in Gegenrichtung) oder <code>Towards</code> (in Fahrtrichtung) angegeben.

<Straßenbelag>	Eine Übersicht über Benennungen der verschiedenen Straßenbeläge finden Sie in Kapitel 6.1 im Dokument REFERENZ DATENBASIS SCNX
<Reibbeiwert>, <Rauigkeit>	Diese Parameter geben Informationen über die Straßenoberfläche an die Fahrdynamik weiter.
<Bebauungstyp>	Straßen können durch verschiedene Bebauungen ausgeschmückt werden. Unter Bebauungen fallen in SILAB sowohl Fahrbahnmarkierungen (Linien) als auch Hecken, Waldbänder, Mauern, Leitplanken und Pfosten. Eine Vielzahl von Bebauungen ist bereits vordefiniert und im Lieferumfang von SILAB enthalten. Eine Übersicht über diese Bebauungstypen finden Sie in Kapitel 6.3 im Dokument REFERENZ DATENBASIS SCNX .
<YOffset>	Verschiebung des Bebauungstyps nach links (<0) bzw. nach rechts (>0) zum exakten Spurübergang (Einheit Meter).

Im letzten Abschnitt werden die Übergänge zwischen den Spuren definiert. Es gibt drei Arten von Bebauungen zwischen zwei Spuren, für die sich die Definition der Bebauungsparameter teilweise unterscheidet: Spurmarkierungen, Objektbänder (z.B. Leitplanken) und einzelne Objekte (z.B. Begrenzungspfosten). Jeder Eintrag beschreibt einen Übergang zwischen zwei Spuren und enthält nacheinander folgende Informationen:

```
LaneTransitions = {(<Spurindex1>, <Spurindex2>, {(<Bebauungstyp>,  
<YOffset>, <Größe>, <Orientierung>, <Abstand>, <Zwischenraum>))}}
```

<Spurindex1>, <Spurindex2>	Angabe der beiden Spuren, zwischen denen der Übergang definiert wird (z.B. 1 und 2). Um die Bebauung der äußersten Fahrbahnrande zu konfigurieren, wird an beiden Stellen der Spurindex der äußersten Spur eingetragen (z.B. 0, 0 für den linken Fahrbahnrand).
<Bebauungstyp>, <YOffset>	s. Beschreibung unter LaneInfos
<Größe>	Breite des Bebauungstyps in Meter
<Orientierung>	Onwards (in Gegenrichtung) oder Towards (in Fahrtrichtung). Die Angabe hat bei Spurmarkierungen und Objektbändern keine Auswirkung und gibt bei einzelnen Objekten die Richtung an, in die die Objekte zeigen sollen.
<Abstand>	Länge einer Fahrbahnmarkierung in Meter, bzw. Abstand zwischen zwei Objekten (keine Auswirkungen bei Objektbändern: 0 eintragen)

<Zwischenraum> Länge des Zwischenraums zwischen zwei Fahrbahn-markierungen in Meter. Für eine durchgezogene Linie wird <Abstand> = 1 und <Zwischenraum> = 0 gesetzt. Dieser Parameter hat bei Objektbändern und einzelnen Objekten keine Auswirkung (0 eintragen).

Info: Eine Vielzahl von Bebauungstypen, Straßenbelägen und Overlay-Texturen ist bereits vordefiniert und im Lieferumfang von SILAB enthalten. Weitere Varianten können projektspezifisch im Konfigurationsabschnitt SCNX Params hinzugefügt werden (siehe Kapitel 5.2 im Dokument **REFERENZ DATENBASIS SCNX**). Der Name der gewünschten Bebauung muss als <Bebauungstyp> eingetragen werden.

Wie bereits zu Beginn erwähnt, müssen Sie diese umfassenden Informationen nicht für jeden Course neu definieren, sondern können alternativ bereits bestehende Querschnittsprofile auf einen Course anwenden. Sie können zum einen Querschnittsprofile, die Sie selbst erstellt haben, in eine include-Datei auslagern und für andere Courses wiederverwenden. Zum anderen können Sie Querschnittsprofile nutzen, die in der Szenario-Datenbank SP_Base oder im SILABDemo-Projektverzeichnis gespeichert sind. In der include-Datei ‚CrossSections.inc‘ unter dem Pfad D:\SILAB\Projects\SILABDemo\includes finden Sie z.B. Querschnittsprofile für Landstraßen, eine Brücke, enge Straßen, einen Tunnel und eine Autobahn. In der Datei ‚SP_Base_CrossSections.inc‘ unter dem Pfad D:\SILAB\Projects\SPDE_Base\LIB\Includes finden Sie weitere vorkonfigurierte Querschnittsprofile. Bitte beachten Sie in diesem Zusammenhang, dass in diesen include-Dateien von SILABDemo und SPDE_Base keine Veränderungen vorgenommen werden dürfen. Wir empfehlen, die gewünschten Dateien zu kopieren, im entsprechenden Projektverzeichnis abzulegen und hier die Datei umzubenennen, um Verwechslungen mit der ursprünglichen Datei zu vermeiden.

Das grundsätzliche Vorgehen besteht daraus, bereits konfigurierte Querschnittsprofile separat in einer include-Datei abzuspeichern. Angenommen, Sie haben bereits eine solche include-Datei mit dem Namen ‚Querschnittsprofile.inc‘ erstellt, in der ein typisches Querschnittsprofil für eine Autobahn und eines für eine Landstraße enthalten sind. Für diese Profile haben Sie die Namen Autobahnprofil und Landstraßenprofil vergeben. Um in der Konfigurationsdatei, in der Sie gerade arbeiten, einem Course nun das Autobahnprofil zuzuweisen, müssen Sie zum einen die include-Datei über eine include-Anweisung in das Skript einbinden und zum anderen an der entsprechenden Stelle in der Course-Definition auf das Autobahnprofil verweisen. Das sieht in der Konfigurationsdatei folgendermaßen aus:

```
### Szenario-Definition

### Querschnittsprofil                               # Einbinden der Datei
include Querschnittsprofile.inc
### /Querschnittsprofil

### Szenario-Modul
```

```
### Courses

### Zuweisung Querschnittsprofil          # Zuweisung eines
CrossSection = Autobahnprofil;            # Querschnittsprofils aus
### /Zuweisung Querschnittsprofil        # der include-Datei

### Lageplan
### /Lageplan

### Höhenprofil
### / Höhenprofil

### Landschaft
### /Landschaft

### Grafische Objekte
### /Grafische Objekte

### Strecken-Events
### /Strecken-Events

### /Courses
```

Code 4. Beispiel für die Anwendung bereits bestehender Querschnittsprofile auf einen Course.

Zur Übersichtlichkeit enthält Code 4 nur die Bestandteile zum Einfügen der include-Datei und der Zuweisung des Querschnittsprofils an der entsprechenden Stelle. Die anderen Bestandteile der Course-Definition sind nur zur Veranschaulichung des Aufbaus als Kommentare eingefügt.

Die include-Anweisung zum Einbinden der Datei, die verschiedene Querschnittsprofile enthält, erfolgt üblicherweise bereits zu Beginn der Szenariodefinition, da sie ggf. für mehrere Courses von dieser include-Datei eingesetzt wird. Die Zuweisung des Querschnittsprofils im Rahmen der Course-Definition ist nun nur noch eine einzige Zeile in der Form

```
CrossSection = <Name des Querschnittsprofils>;
```

Der zugewiesene Name muss mit der Benennung des Querschnittsprofils in der include-Datei übereinstimmen.

5.1.2 Lageplan

Unter dem Lageplan versteht man den Straßenverlauf eines Course in der Ebene, welcher sich aus Geraden einer definierten Länge und Kurven mit definierten Längen und Radien zusammensetzt. Alle Geraden und Kurven summieren sich zu der Gesamtlänge des Course auf.

Die Definition eines Lageplans in der Konfigurationsdatei sieht bspw. folgendermaßen aus:

```
### Lageplan

Straight(200);
Bend(1000, 2000);
Bend(1500, -3000);
Straight(100);
Bend(1000, -2500);
Straight(200);
Bend(1000, 2700);

# Gerade
# Rechtskurve
# Linkskurve
# Gerade
# Linkskurve
# Gerade
# Rechtskurve

### /Lageplan
```

Code 5. Beispiel für die Definition eines Lageplans.

Die Definition des Lageplans besteht aus der Aneinanderreihung von Kurven und Geraden. Die Stücke erscheinen bei der Simulationsfahrt genau in der Reihenfolge, wie Sie in diesem Abschnitt der Konfigurationsdatei definiert haben.

Geraden werden mit dieser Anweisung definiert:

```
Straight(<Länge in m>);
```

Kurven werden mit dieser Anweisung definiert:

```
Bend(<Länge in m>, <Radius in m>);
```

<Radius in m> Ein negativer Kurvenradius entspricht einer Linkskurve, ein positiver entspricht einer Rechtskurve.

Der Course in Code 5 hat eine Gesamtlänge von 5000 m und besteht aus drei Geraden, zwei Linkskurven und zwei Rechtskurven.

5.1.3 Höhenprofil des Straßenverlaufs

Zusätzlich zum Straßenverlauf des Course in der Ebene wird das Höhenprofil, dem die Straße folgt, in einem separaten Abschnitt der Course-Definition festgelegt. Das Höhenprofil der Straße wird unabhängig vom Lageplan definiert.

Im Folgenden sehen Sie ein Beispiel für die Definition des Höhenprofils einer Straße, die 5 km lang ist.

```
### Höhenprofil
HeightProfile =
{
    ( 0.0, 500.0, 4000),
    ( 0.5, 400.0, 4000),
    ( 0.0, 600.0, 4000),
    (-0.8, 500.0, 4000),
    ( 1.0, 800.0, 4000),
    ( 0.0, 600.0, 4000),
    (-0.6, 400.0, 4000),
    ( 0.0, 300.0, 4000),
    ( 0.4, 700.0, 4000),
    ( 0.0, 200.0, 4000)
};
### /Höhenprofil
```

Code 6. Beispiel für die Definition eines Höhenprofils.

Das Höhenprofil besteht aus einer Abfolge von Segmenten, die entweder eben sind oder eine Steigung bzw. ein Gefälle haben. In der Definition entspricht jedes Element in runden Klammern einem solchen Segment. Die Definition setzt sich folgendermaßen zusammen:

```
HeightProfile= {( <Steigung in %>, <Länge des Segments in m>, <Ausrundungshalbmesser> )};
```

<Steigung in %> Anstieg der Straßenhöhe nach 100 m entlang der Straße in %.

Ein positiver Wert entspricht einem Anstieg und ein negativer Wert einem Gefälle.

Bspw. bedeutet eine Steigung von 0.5%, dass die Straßenhöhe nach 100 m um 0.5 m ansteigt.

<Ausrundungshalbmesser> Würde SILAB einfach nur die einzelnen Segmente aneinanderfügen, würden Knicke an den Stellen im Straßenverlauf entstehen, an denen sich die Steigung ändert. Im Straßenbau werden daher sog. Ausrundungen an diesen Stellen vorgeschrieben. Die Form und Größe dieser Ausrundungen, die SILAB automatisch einfügt, können mit dem Parameter <Ausrundungshalbmesser> beeinflusst werden. In den allermeisten Fällen können Sie hier einfach den Wert 4000 eintragen.

Die Gesamtlänge aller Segmente des Höhenprofils muss der Gesamtlänge des Straßenverlaufs in der Ebene (Lageplan) entsprechen.

Das erste und letzte Segment eines Höhenprofils sollten immer die Steigung 0% haben.

5.1.4 Umgebungslandschaft

In SILAB erfolgt die Gestaltung der Landschaft, von der die Straße umgeben ist, durch Festlegen sogenannter Landschaftsgeneratoren. Mit diesem Mechanismus wird die Landschaft auf der rechten und linken Straßenseite einzeln entsprechend des ausgewählten Landschaftstyps und den von Ihnen gewählten Parametereinstellungen modelliert. Sie können die Umgebung mit verschiedenen, vordefinierten Landschaftstypen generieren lassen, z.B. ländliche Gegend, Wald, Weinberge, Alleen und bewohntes Gebiet. Der Landschaftsgenerator enthält ein Höhenmodell, mit dem Hügel und Täler der Landschaft modelliert werden und platziert Objekte in der Landschaft, z.B. Bäume und Häuser. Beispielsweise modelliert der Landschaftsgenerator ‚Farmed‘ eine ländliche Gegend mit Feldern, Feldwegen und anderen für diese Landschaft typischen Elementen. Somit müssen SILAB-Anwender nicht an jeder Stelle der Landschaft das Höhenmodell festlegen sowie einzelne Objekte entlang der Straße platzieren. Mit den Parametern des Landschaftsgenerators können Sie das Aussehen der Landschaft beeinflussen. Sie können festlegen, wie hügelig die Landschaft ist und wie viele Elemente eines bestimmten Objekts (z.B. Bäume) durchschnittlich pro Flächeneinheit verteilt werden sollen. Aus diesen Angaben erzeugt SILAB automatisch während der Simulation eine ansprechende umgebende Landschaft.

Wird auf einer Straßenseite kein Landschaftstyp angegeben, so ist die Landschaft auf dieser Seite weitgehend flach und es werden keine Objekte platziert.

Code 7 zeigt ein Beispiel für eine Definition der Umgebungslandschaft in der Konfigurationsdatei:

```
### Landschaft

LandscapeTypeLeft Wooded
{
    Height = 3;
    Ripple = 0.7;
    Offset = 5;
    Gradient = -0.3;

    OverlayDensity = 0.8;
    TreeDensity = 90;
    MinObjdist = 15;
    MaxObjdist = 60;
};

LandscapeTypeRight Farmed
{
    Height = 2.5;
```

```
# Bewaldetes Gebiet auf der linken # Straßenseite
# Parameter des Höhenmodells der # Landschaft

# Verteilung der Landschaftsobjekte

# Ländliche Gegend auf der rechten # Straßenseite
# Parameter des Höhenmodells der # Landschaft
```



```

Ripple = 0.1;
Offset = 0;

OverlayDensity = 0.85;      # Verteilung der Landschaftsobjekte
MaxObjdist = 350;
FieldSetback = (8, 0.5);
};

### /Landschaft

```

Code 7. Beispiel für die Definition der Umgebungslandschaft.

Im Folgenden werden der Aufbau und die Bestandteile der Landschafts-Definition erläutert. Detailliertere Informationen zur Grammatik der Skriptsprache, mit der die Umgebungslandschaft generiert wird, sowie zu den speziellen Parametereinstellungen für die verschiedenen Landschaftstypen finden Sie in Kapitel 2.5.13 im Dokument **REFERENZ DATENBASIS SCNX**.

Die Definition wird in zwei Abschnitte für die linke und rechte Straßenseite unterteilt. Mit den Schlüsselwörtern `LandscapeTypeLeft` bzw. `LandscapeTypeRight` werden die Landschaftsgeneratoren aktiviert.

```

LandscapeTypeLeft <Landschaftstypname>
LandscapeTypeRight <Landschaftstypname>

```

<Landschaftstypname> Es gibt 24 vordefinierte Landschaftstypen, deren Aussehen je durch einige Parameter beeinflusst werden kann (s. Tabelle 1)

Pro Straßenseite werden im ersten Abschnitt die Parameter des Höhenmodells der Landschaft definiert (unabhängig vom Landschaftstyp). Die Angabe dieser Parameter ist optional. Ohne Angabe der Parameter bleibt die Landschaft flach. Abbildung 2 veranschaulicht die Bedeutung der folgenden Parameter.

Height	Amplitude der Landschaftshöhen in m (maximale Tiefe von Senken und die maximale Höhe von Hügeln). Sinnvolle Werte liegen zwischen 0 und 10.
Ripple	Frequenz der Schwingung, d.h. Abwechseln von Hügeln und Senken. Sinnvolle Werte liegen zwischen 0 und 4.
Offset	Offset der Landschaftshöhe in Bezug auf die Höhe der Straße in Metern. Sinnvolle Werte liegen zwischen -10 und 10.
Gradient	Steigung, mit der die gesamte Landschaft mit zunehmendem Abstand von der Straße ansteigt oder abfällt. Der Wert gibt den Höhenunterschied in Metern pro Meter Abstand von der Straße an. Sinnvolle Werte liegen zwischen -0.5 und 0.5.
Roughness	Maximale Höhe von kleinen Unebenheiten in der Landschaft in Metern.

Entfernung von der Straße in Metern, bis zu der noch Unebenheiten (entsprechend Height, Ripple, und Roughness) dargestellt werden. Im größeren Abstand wird nur noch eine relativ glatte Landschaft erzeugt, die nur grob der Höhe der Straße folgt. Der Standardwert ist 200. Sinnvolle Werte liegen zwischen 100 und 500.

MaxHillDist

Im nächsten Abschnitt werden Parameter definiert, die sich auf die Verteilung der Landschaftsobjekte beziehen. Für die meisten Landschaftstypen gibt es über die folgenden Parameter hinaus zusätzliche spezielle Parameter, die Sie in Kapitel 2.5.13 im Dokument **REFERENZ DATENBASIS SCNX** finden.

Dichte von Overlay-Feldern. Felder, Wiesen oder Waldboden werden abhängig vom Landschaftstyp direkt auf die Landschaft gezeichnet. Wertebereich 0 bis 1, Default-Wert 0.2. (Wenn der Wert 1 angegeben wird, wird die komplette Landschaftsoberfläche mit Overlay-Feldern gefüllt.)

OverlayDensity

Anzahl der Bäume, die im Durchschnitt auf einer Fläche von 10.000 m² (entspricht 1 Hektar) verteilt werden sollen.

TreeDensity

Minimaler Abstand der Landschaftsobjekte zur Straße in Metern, Wertebereich von -2000 m bis 2000 m, Default-Wert : 0 m.

MinObjDist

Maximaler Abstand der Landschaftsobjekte zur Straße in Metern, Wertebereich von 0 m bis 2000 m, Default-Wert : 2000 m.

MaxObjDist

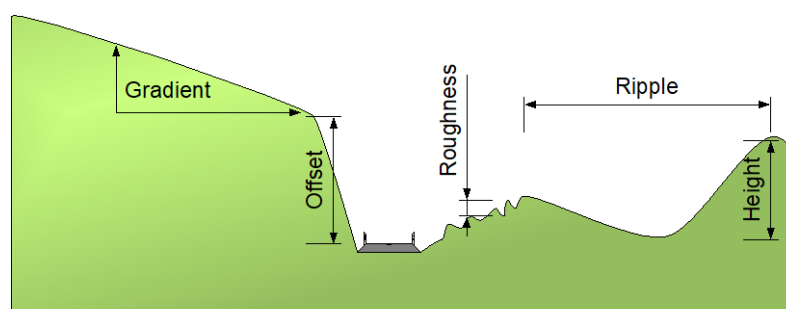


Abbildung 2. Landschafts-Parameter.

Zu extreme Einstellungen im Höhenmodell sehen sehr unrealistisch aus und können auch zu einer unruhigen Darstellung der Landschaft führen.

Sie können durch eine Erweiterung der SCN-X-Konfiguration zusätzlich auch eigene Landschaftstypen, in denen andere grafische Objekte platziert werden, definieren. Dies wird in Kapitel 5.2.4 im Dokument **REFERENZ DATENBASIS SCN-X** näher beschrieben.

Tabelle 1. Übersicht über mitgelieferte Landschaftstypen.

Landschaftstyp	Beschreibung
Farmed	Korn- und Maisfelder, die durch Feldwege erschlossen sind
Meadows	Weiden, die durch Zäune begrenzt und durch Feldwege erschlossen sind. Auf den Weiden können optional Bäume und/oder Tiere platziert werden.
Forest	Nadelwald: Am Rand des Waldes zur Straße hin werden zusätzlich Unterholz und bodendeckende Pflanzen platziert.
LeafForest	Laubwald: Am Rand des Waldes zur Straße hin werden zusätzlich Unterholz und bodendeckende Pflanzen platziert.
AncientVillage	Dorf mit Fachwerkhäusern und kleineren Bäumen
CountryVillage	Dorf mit Steinhäusern und kleineren Bäumen
Suburb	Vorort mit modernen Wohnhäusern und kleineren Bäumen
Inhabited	Bungalow-Siedlung mit Personen und kleineren Bäumen
Vineyard	Weingarten oder Weinberg
Wooded	Baumbeständiges Gebiet, optional können einzelne Häuser platziert werden
FruitTree	Obstbaugelände, in dem die Obstbäume auf einer Wiese stehen
Alley1	Allee aus kleineren Straßenbäumen
Alley2	Allee aus schlanken, hohen Bäumen
Hedge	Hecke aus kleinen Bäumen und Büschen
Grassland	Wiese aus verschiedenen Gräsern und Blumen
Meadow	Weide oder frisch gemähte Wiese aus Gräsern
DryMeadow	Ausgetrocknete Weide aus Gräsern
RoadsideBush	Reihe von Büschen entlang der Straße: Dieser Landschaftsgenerator wird beispielsweise verwendet, um den Mittelstreifen einer Autobahn zu begrünen.
Tunnel	Platziert eine Reihe von Tunnelabschnitten entlang der Straße, die breit genug für eine zweispurige Landstraße oder für eine Fahrtrichtung einer Autobahn mit zwei Fahrspuren sind. In regelmäßigen Abständen werden spezielle Tunnelabschnitte (z.B. Nothaltebucht, Notrufsäule, Fluchtweg) integriert.
GroundFog	Feld von Nebelschwaden

5.1.5 Grafische Objekte

Neben der allgemeinen, großflächigen Ausgestaltung der Landschaft mittels Landschaftsgeneratoren können Sie auch gezielt einzelne grafische Objekte in der Landschaft platzieren. Zu den grafischen Objekten zählen Verkehrsschilder, Straßenpfosten, Bäume, Häuser usw.

Innerhalb der Course-Definition werden die grafischen Objekte in einem Abschnitt festgelegt, der sich auf der gleichen Ebene wie die Landschaftsdefinition befindet. Dieser Abschnitt sieht beispielsweise folgendermaßen aus:

```
### Grafische Objekte
Objects =
{
    (house.Windmill, 2550, 100),          # Windkraftanlage
    (environment.tree.tree1, 2650, -110), # Baum
    (house.house1, 2750, 120)             # Haus
};
### /Grafische Objekte
```

Code 8. Beispiel für die Definition grafischer Objekte.

In dem Beispiel in Code 8 lernen Sie eine von vier Versionen kennen, um Objekte in der Landschaft zu platzieren. In den anderen drei Versionen wird die Definition noch um weitere Parameter ergänzt. Diese ergänzenden Informationen finden Sie in Kapitel 2.5.6 im Dokument [REFERENZ DATENBASIS SCNX](#).

Alle einzeln platzierte grafische Objekte werden als Elemente der Menge `Objects = {...}`; aufgeführt. Jedes Element entspricht einem Objekt und hat (im einfachsten Fall) diese Form:

(`<Objektname>`, `<Streckenmeter>`, `<Abstand>`)

<code><Objektname></code>	Dieser Parameter bezieht sich auf Dateien, in denen SILAB die Informationen zu einem grafischen Objekt finden kann. Im Lieferumfang ist eine große Bibliothek mit grafischen Objekten enthalten. Sie befinden sich im Unterordner <code>,lib\graphics\default\models'</code> (Modellordner) des SILAB Installationsverzeichnis. Der <code><Objektname></code> ist der Pfad zu einer solchen Datei relativ zum Modellordner. Das Zeichen <code>,</code> wird durch einen Punkt ersetzt, die Dateinamenserweiterung <code>*.sig</code> oder <code>*.sge</code> wird weggelassen. Z.B. ist die Windkraftanlage aus Code 8 in der Datei <code>,lib\graphics\default\models\house\Windmill.sig'</code> definiert.
<code><Streckenmeter></code>	legt die Position des Objekts entlang der Straße fest, gemessen in Metern vom Beginn der Straße aus.
<code><Abstand></code>	legt den gerichteten Abstand des Objekts (in Metern) von der Mittellinie der Straße aus fest. Negative Abstände geben Positionen links von der Straßenmitte, positive Abstände rechts davon an.

Der simulierte Verkehr reagiert nicht auf Verkehrsschilder, die als grafisches Objekt platziert wurden. Falls der simulierte Verkehr auf die Schilder reagieren soll, muss eine Verkehrsregel definiert werden. Dies erfolgt über den ‚TrafficRules‘-Abschnitt, der der Course-Definition untergeordnet wird. Mehr Informationen zu diesem Thema finden Sie in Kapitel 2.5.9 im Dokument **REFERENZ DATENBASIS SCNX**.

5.1.6 Strecken-Events

Strecken-Events werden in SILAB eingesetzt, um automatisch bestimmte Ereignisse auszulösen. Es handelt sich dabei um unsichtbare Markierungen auf den Spuren der Strecke. Wenn der Fahrer eine solche Markierung überfährt, wird ein bestimmtes Ereignis ausgelöst. Der Punkt enthält genaue Informationen darüber, welche Ereignisse in welcher Form beim Überfahren ausgelöst werden sollen. Das bedeutet, dass Sie in der Konfigurationsdatei definieren können, dass beim Überfahren von Strecken-Events bestimmte Veränderungen von Parametern und DPU-Eingängen erfolgen. Durch Strecken-Events können Sie beispielsweise Soundausgaben, Umgebungs- oder Wetterbedingungen oder Ereignisse in der Fußgängersimulation aktivieren.

Strecken-Events sind ein wichtiger Mechanismus, um Simulatorfahrten reproduzierbar zu machen. Wenn Sie beispielsweise einen Versuch durchführen, in dessen Verlauf DPU-Parameter szenarioabhängig verändert werden sollen, dann sollten Sie dazu Strecken-Events einbinden, anstatt die Veränderungen interaktiv über die Operatoroberfläche auszulösen.

Zum Einbinden eines Strecken-Events wird die Konfigurationsdatei an verschiedenen Stellen bearbeitet:

- Einbinden der DPU, deren Eingänge über die Strecken-Events gesteuert werden sollen.
- Einbinden der DPUSCNXHedgehogKiller: Diese DPU registriert Strecken-Events von bestimmten Kategorien, die in der DPU-Konfiguration von Ihnen festgelegt werden. Die DPU besitzt Ausgänge, die beim Überfahren von Strecken-Events ihren Wert ändern.
- Verknüpfung der Ausgänge der DPUSCNXHedgehogKiller mit den Eingängen der zu steuernden DPUs
- Definition des Strecken-Events innerhalb der Course-Definition

Diese verschiedenen Schritte zum Einbinden eines Strecken-Events werden nun anhand eines Beispiels durchlaufen, in dem mit Strecken-Events ein Navigationssystem realisiert werden soll. Das Navigationssystem hat drei Sprachmeldungen, die jeweils in einer WAV-Datei gespeichert sind:

- gerade.wav: „Bitte dem Straßenverlauf folgen.“
- links.wav: „Jetzt links abbiegen.“
- rechts.wav: „Jetzt rechts abbiegen.“

Die Bezeichnung für ein Strecken-Event innerhalb der Konfigurationsdatei ist Hedgehog.

SCHRITT 1:

Die WAV-Dateien werden durch die DPUSNDXWavPlayer abgespielt. Die dazugehörige DPU-Konfiguration würde folgendermaßen aussehen:

```
DPUSNDXWavPlayer NaviAnsagen
{
  Computer = {SND};
  Index = 10;
  SoundPath = "\\Projects\\MeinProjekt\\";
  Sound01 = "gerade.wav";
  Sound02 = "links.wav";
  Sound03 = "rechts.wav";
};
```

verwendete DPU und Name der DPU-# Instanz
Name des Soundrechners
Index der DPU
Speicherort der WAV-Dateien
Benennung der Navigationsansagen

Code 9. Beispiel für die DPU-Konfiguration der DPUSNDXWavPlayer.

SCHRITT 2:

Jede DPU, die über Strecken-Events gesteuert werden soll, bekommt eine Instanz der DPUSCNXHedgehogKiller vorgeschaltet. Diese DPU sammelt in jedem Simulationsschritt die überfahrenen Events einer bestimmten Kategorie auf und stellt deren Argumente an ihren Ausgängen zur Verfügung. Aufgrund dessen wird diese DPU ebenso in die DPU-Konfiguration eingebunden. In diesem Rahmen wird festgelegt, dass die Strecken-Events für die Navigationsansagen der Kategorie ‚Navi‘ zugeordnet werden. Diese Angabe hat zur Folge, dass die DPUSCNXHedgehogKiller während der Simulationsfahrt die Strecken-Events der Kategorie ‚Navi‘ registriert. Dieser Abschnitt sieht in der DPU-Konfiguration wie folgt aus:

```
DPUSCNXHedgehogKiller NaviEvents
{
  Computer = {SCN};
  Index = 20;
  Family = "Navi";
  Name1 = "WavIndex";
  OutModel = (MonoFloP, 500);
};
```

verwendete DPU und Name der DPU-# Instanz
Name des Datenbasis-Rechners
Index der DPU
Zuordnung zur Kategorie „Navi“
Werte des Parameters Name1 werden # an Ausgang Out1 geliefert

Code 10. Beispiel für die DPU-Konfiguration der DPUSCNXHedgehogKiller.

SCHRITT 3:

Die Verknüpfung zwischen den beiden DPU-Instanzen NaviAnsagen und NaviEvents muss in den Abschnitt der DPU-Konfiguration hinzugefügt werden, in dem die DPU-Verknüpfungen (Connections) definiert werden. Diese Verknüpfung gibt an, dass der Ausgang der Instanz NaviEvents, an

dem die aktivierten Werte beim Überfahren des Strecken-Events liegen, mit dem Eingang der Instanz NaviAnsagen verbunden wird, worauf eine bestimmte WAV-Datei abgespielt wird. Eine solche Verknüpfung sieht folgendermaßen aus:

```
Connections =
{
    NaviEvents.Out1 -> NaviAnsagen.Play
};
```

Code 11. Beispiel für die Verknüpfung von DPU-Instanzen zur Erstellung von Strecken-Events.

SCHRITT 4:

In der Course-Definition in der Konfigurationsdatei wird ein Unterabschnitt zur Definition der Streckenevents eingefügt, der folgende Form hat:

```
### Strecken-Events
Hedgehogs =
{
    (150, Normal, 1, Navi, WavIndex, 1),
    (250, Normal, 1, Navi, WavIndex, 2)
};
### /Strecken-Events
```

Code 12. Beispiel für die Definition von Strecken-Events.

Strecken-Events werden als Elemente der Menge Hedgehogs = { . . . }; aufgeführt. Jedes Element entspricht einem Strecken-Event und hat diese Form:

(<Streckenmeter>, Normal Reverse, <Spurnummer>, <Familie>, <Hedgehog-Argument>)	
<Streckenmeter>	Der Streckenmeter legt die Position des Strecken-Events entlang der Straße fest, gemessen in Metern vom Beginn der Straße aus.
Normal Reverse	Das Strecken-Event gilt als überfahren, wenn der Fahrer in seine eigene Fahrtrichtung (Normal) oder auf der Gegenspur (Reverse) fährt.
<Spurnummer>	Die Nummer der Spur, auf der das Strecken-Event liegt (die Spurnummern werden im Querschnittsprofil unter LaneInfos festgelegt, s. Abschnitt 5.1.1)
<Familie>	Der Name der Familie/Kategorie, zu der das Strecken-Event gehört (Family = "Navi" in Schritt 2)
<Hedgehog-Argument>	Jedes Hedgehog-Argument besteht aus: <Name>, <Wert>
<Name>	Der Name des Parameters wird in der DPU-Instanz der DPUSCNXHedgehogkiller definiert (Name1 = "WavIndex" in Schritt 2)

<Wert> Der Wert gibt an, welcher Wert des Parameters ausgegeben werden soll (im Beispiel: Wert von WavIndex; Wert 1 bedeutet, dass die Ansage ‚gerade.wav‘ abgespielt werden soll, 2 bedeutet ‚links.wav‘ und 3 ‚rechts.wav‘)

In dem Beispiel in Code 12 bedeutet das erste Element dementsprechend: Sobald der Fahrer bei Streckenmeter 150 (<Streckenmeter>) auf Spur 1 (<Spurnummer>) in Fahrtrichtung dieser Spur (Normal) über die Markierung fährt, liefert die DPU-Instanz NaviEvents am Ausgang Out1 (dem Parameter ‚WavIndex‘ wurde in der DPU der Ausgang 1 zugewiesen) den Wert 1. Der Wert 1 entspricht der Navi-Ansage mit der WAV-Datei ‚gerade.wav‘.

Da überfahrene Strecken-Events von der DPUSCNXHedgehogKiller als Ausgänge zur Verfügung gestellt werden, können sie auch in der Aufzeichnungsdatei als Messwerte in eine Spalte geschrieben werden. Um eine spätere Aufteilung und Auswertung der Daten erheblich zu vereinfachen, können Sie damit bestimmte Abschnitte in Ihrem Streckennetz markieren.

5.2 Gestaltung von Areas

Zur Gestaltung der komplexeren Spurgeometrie von Areas wird das Programm SILABAEEdit verwendet, welches im Lieferumfang von SILAB enthalten ist. SILABAEEdit ist ein grafischer Editor, mit dem Sie folgende Bestandteile einer Area komplett ohne Skriptsprache gestalten können:

- Festlegen des Layouts: Mit dem Zeichnen von Layout-Skizzen wird das grobe Aussehen der Straßengeometrie mit Linien, Kreisen und Kreisbögen entworfen.
- Festlegen der Querschnitte: Dialogfenster, mit den Spurnanzahl, Fahrtrichtungen, Spurbreiten und Höhenprofil festgelegt werden
- Verbinden der Querschnitte mit Fahrspuren
- Grafische Gestaltung: Aufstellen von Verkehrsschildern, Platzierung von grafischen Objekten, Einfügen von Objektzeilen
- Einfügen von Verkehr und Fußgängern, Verhaltensdefinition, Einfügen von Streckenevents
- Festlegen der Verkehrssteuerung

Im **HANDBUCH SILABAEEDIT** finden Sie eine separate, detaillierte Anleitung für die Erstellung von Areas mit SILABAEEdit.

5.2.1 Einbinden von Areas in die Konfigurationsdatei

Areas, die mit SILABAEEdit erstellt wurden, können entweder als Streckenstück oder als gesamtes Szenario-Modul in die Konfigurationsdatei eingebunden werden. SILABAEEdit kann automatisch Skript-Dateien im CFG-Format für die Areas erzeugen, die über die include-Funktion in die übergeordnete Konfigurationsdatei integriert werden. Für den Export der Area aus SILABAEEdit nutzen Sie die Menüfunktion: ‚Datei → Exportieren → SILAB...‘ oder nutzen Sie die Tastenkombination ‚Strg + E‘.

Wenn Sie eine Area als Streckenstück exportieren möchten, um es daraufhin in der Konfigurationsdatei mit anderen Area- und/oder Course-Streckenstücken zu einem Streckennetz zu verknüpfen, geben Sie in das Export-Dialogfenster von SILABAEEdit (s. Abbildung 3) den Projektpfad, einen beliebigen Namen für die CFG-Datei und für das Area-Streckenstück ein. Weiterführende Informationen zum Exportieren von Areas aus SILABAEEdit finden Sie in Kapitel 4 im [HANDBUCH SILABAEEDIT](#).

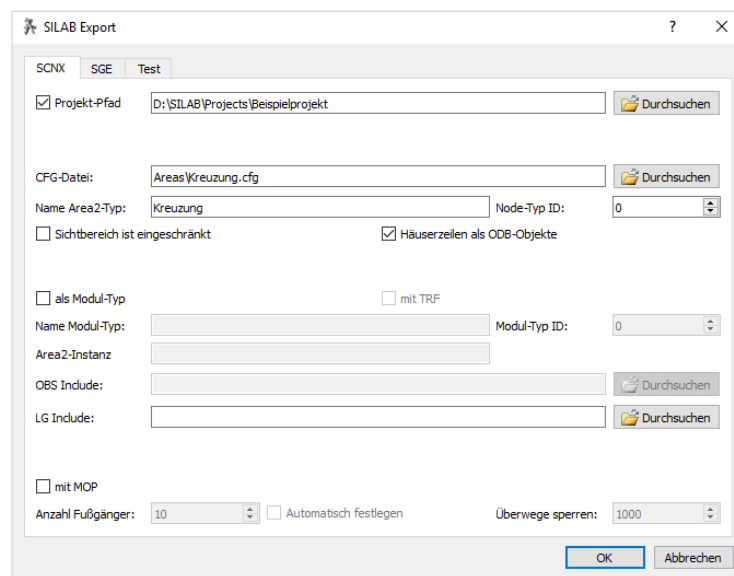


Abbildung 3. Export einer Area als Streckenstück aus SILABAEEdit.

Diese Konfigurationsdatei bzw. include-Datei, die durch den Export der Area erstellt wird, muss daraufhin in die übergeordnete Konfigurationsdatei eingebunden werden, welche die Definition der Course-Streckenstücke sowie alle weiteren Konfigurationsabschnitte enthält. Das Einfügen der Areas erfolgt auf der gleichen Unterabschnitt-Ebene wie die Course-Definitionen. Wenn die include-Datei mit der Area-Definition in dem gleichen Ordner wie die Haupt-Konfigurationsdatei gespeichert wurde, wird hinter der include-Anweisung nur der Dateiname angegeben (z.B. include Kreuzung.cfg). Befindet sich die include-Datei in einem Unterordner des Projektordners, muss der Dateipfad angegeben werden (z.B. include parts\Kreuzung.cfg). Im folgenden, beispielhaften Auszug aus der Szenariodefinition einer Konfigurationsdatei sehen Sie, wie Areas über die include-Funktion eingefügt werden.

```
### Definition der Streckenstücke
```

```
### Courses
define Course Course1
{
# siehe Abschnitt 5.1
};

define Course Course2
{
# siehe Abschnitt 5.1
};
### /Courses

### Einfügen der Areas
include Areas\Kreuzung.cfg
include Areas\Kreisverkehr.cfg
### /Einfügen der Areas

### /Definition der Streckenstücke
```

Code 13. Beispiel für eine Szenariodefinition, in die über include-Anweisungen Area-Streckenstücke eingebunden werden.

Die Verknüpfung von Course- und Area-Streckenstücken zu einem Streckennetz erfolgt nach dem Einbinden der Areas in die Konfigurationsdatei in einem separaten Abschnitt und unter Verwendung der Skript-Sprache. Sie wird in Kapitel 6 beschrieben.

Der Export einer Area als komplettes Szenariomodul wird empfohlen, wenn eine Area ein abgeschlossenes Modul mit Verkehrs- und Fußgängerdefinition darstellt, z.B. einen Häuserblock. Durch den Export als Szenariomodul werden Verkehrs- und Fußgängerdefinition automatisch in dieses Modul integriert. In dem entsprechenden Dialogfenster in SILABAEEdit (s. Abbildung 4) müssen in diesem Fall noch weitere Angaben gemacht werden, als bei dem Export einer Area als Streckenstück. Wählen Sie die beiden Kästen ‚als Modul-Typ‘ und ‚mit TRF‘ aus. Es wird ein Name (Name Modul-Typ) und eine ID (Modul-Typ ID) für das Szenario-Modul vergeben. Das Modul enthält ein Area-Streckenstück mit dem Namen, der unter ‚Name Area2-Typ‘ angegeben wird und der dazugehörigen ‚Node-Typ ID‘. In dem Feld ‚Area2-Instanz‘ geben Sie den Namen ein, den die Instanz dieses Streckenstücks erhalten soll (Kapitel 6 enthält eine Erläuterung zur Instanziierung von Streckenstücken). Wenn Sie den Kasten ‚mit MOP‘ auswählen, können Sie die Anzahl der Fußgänger festlegen, die für diese Area zur Verfügung stehen. Der Standardwert ‚1000‘ unter ‚Überwege sperren‘ gibt an, dass die Fußgängerüberwege nur durch die Ampelschaltung gesperrt sind. Wird die Area mit den oben beschriebenen Einstellungen exportiert, genügt zur Integration in eine Konfigurationsdatei eine einzelne include-Anweisung. Diese enthält die Modul-Definition sowie die Verkehrsdefinition der Area (Code 14). Weiterführende Informationen zum Exportieren von Areas als Szenariomodul aus SILABAEEdit finden Sie in Kapitel 4 im [HANDBUCH SILABAEEDIT](#).

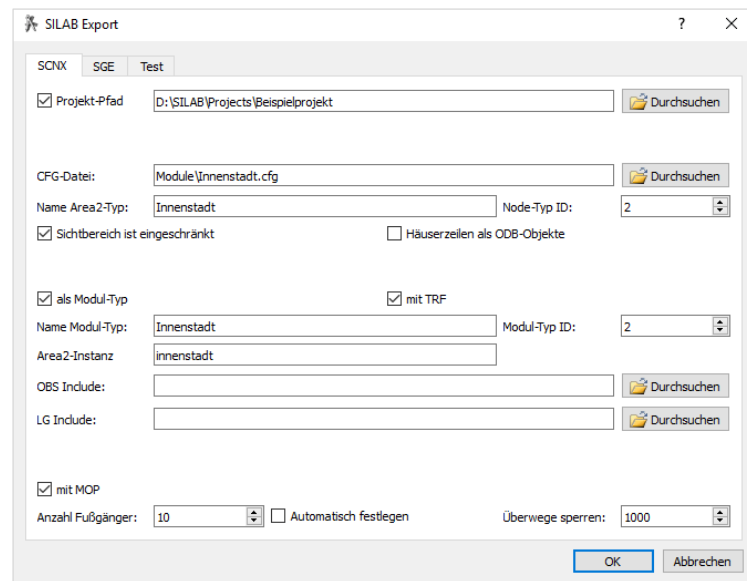


Abbildung 4. Export einer Area als Szenario-Modul aus SILABEdit.

Das exportierte Szenariomodul wird im nächsten Schritt als include-Datei in die übergeordnete Konfigurationsdatei integriert. Dies erfolgt auf der gleichen Ebene im Skript, auf der die weiteren Szenario-Module definiert werden. Code 14 enthält ein Beispiel, in dem zwei Areas als Szenario-Module in die Konfigurationsdatei eingefügt werden.

```
### Szenario-Definition
SILAB SCN
{
  ### Szenario-Modul X
  define Module ModulX
  {
    # siehe Kapitel 5,6,7
  };

  ### Einfügen von Areas als Szenario-Module
  include Module\Innenstadt.cfg
  include Module\Dorf.cfg
  ### /Einfügen von Areas als Szenario-Module

  ### /Szenariodefinition
```

Code 14. Beispiel für eine Szenariodefinition, in die über include-Anweisungen Area-Module eingebunden werden.

Die Verknüpfung von Modulen, die mit SILABEdit erstellt wurden, und Modulen, die in der Konfigurationsdatei definiert wurden, zu einer Karte erfolgt unter Verwendung der Skript-Sprache in einem separaten Abschnitt der Konfigurationsdatei ('Erzeugung einer Karte aus Szenario-Modulen'). Kapitel 8 beschreibt, wie eine Karte aus Szenariomodulen erzeugt wird.

5.3 Nachbau realer Strecken

Mit SILAB können Sie reale Strecken nachbauen. Der erste Schritt zum Nachbau einer realen Strecke besteht aus ihrer Zerlegung in Course- und Area-Streckenstücke. In diesem Zusammenhang sollten Streckenstücke mit einer Länge ab ca. 0.5 bis 1 km, welche ein durchgängiges Querschnittsprofil und keine Abzweigungen besitzen, als Course definiert werden. Alle anderen Streckenstücke werden mit SILABAEEdit als Area modelliert.

Für die konkrete Umsetzung des Nachbaus haben Sie im Idealfall Zugriff auf den Trassierungsplan der Streckenstücke. Dieser Plan enthält Angaben über Längen, Radien und Übergangsbögen (sogenannte Klothoide) der Strecke. Diese Angaben können Sie für die Definition der Streckenstücke in SILAB übernehmen. Wenn Ihnen der Trassierungsplan der Strecke nicht vorliegt, können Sie die Elemente des Lageplans (Straßenverlauf in der Ebene) z.B. mit Google Maps abschätzen. Die Abschätzung des Höhenplans gestaltet sich mit dieser Methode jedoch schwieriger. Dieser kann bspw. aus GPS-Daten nachgebildet werden. Außerdem sollte sich die Definition der Streckenstücke an offiziellen Vorgaben zum Straßenbau orientieren. Die Forschungsgesellschaft für Straßen- und Verkehrswesen (FGSV) hat folgende technische Regelwerke für den Straßenentwurf veröffentlicht:

- Richtlinien für die Anlage von Stadtstraßen (RASt, Ausgabe 2006)
- Richtlinien für die Anlage von Landstraßen (RAL, Ausgabe 2012)
- Richtlinien für die Anlage von Autobahnen (RAA, Ausgabe 2008)

Diese Richtlinien enthalten Vorschriften zur Gestaltung von Straßenquerschnitten, Lage- und Höhenplänen und Knotenpunkten für die verschiedenen Straßentypen.

Für die Modellierung von Area-Streckenstücken bietet SILABAEEdit die Möglichkeit, ein Modell-Bild als Hintergrund einzubinden und dieses maßstabsgerecht zu skalieren. Geeignete Modell-Bilder können z.B. aus Screenshots von Google Maps gewonnen werden. An diesem Hintergrundbild können Sie sich während des Entwurfs von Layout-Linien, Querschnitte, Spuren usw. orientieren.

Eine Alternative zum oben beschriebenen Vorgehen stellt der mit SILAB 7.0 in SILABAEEdit verfügbare OpenStreetMap Import dar. Mithilfe dieses Moduls ist es möglich, Streckenabschnitte auf einer Karte auszuwählen und deren Geometrie vollautomatisch in eine Area zu importieren.

In Kapitel 5.1 im **HANDBUCH SILABAEEDIT** finden Sie Informationen zur Nutzung von Modellbildern zum Entwurf von Area-Streckenstücken.

6 ERZEUGUNG EINES STRECKENNETZES AUS STRECKENSTÜCKEN

Nachdem Sie verschiedene Streckenstücke erstellt haben, werden diese miteinander zu einem Streckennetz verbunden. Dieser Abschnitt Erzeugung des Streckennetzes befindet sich in der Konfigurationsdatei auf der gleichen Ebene wie die Definition der Streckenstücke und die Verkehrsdefinition. Diese drei Abschnitte werden der Szenario-Modul-Definition untergeordnet. Das Streckennetz wird zusammen mit der Verkehrsdefinition für dieses Streckennetz als Szenario-Modul bezeichnet.

```
### Szenario-Modul

### Definition der Streckenstücke
### /Definition der Streckenstücke

### Erzeugung des Streckennetzes
### /Erzeugung des Streckennetzes

### Verkehrsdefinition
### /Verkehrsdefinition

### /Szenario-Modul
```

Code 15. Aufbau der Definition eines Szenario-Moduls.

Jedes Streckenstück besitzt Anschlüsse, mit denen es mit anderen Streckenstücken verbunden werden kann. Ein Course hat immer zwei Anschlüsse, die mit ‚Begin‘ und ‚End‘ bezeichnet werden. Bei Areas werden die Anschlüsse als ‚Port‘ bezeichnet. Die Anzahl der Ports pro Area hängt von der Streckengeometrie ab. Eine klassische Kreuzung hat z.B. vier und eine T-Kreuzung drei Ports. SILABAEEdit vergibt die Namen der Ports automatisch, sie werden immer mit ‚Port1‘, ‚Port2‘, ‚Port3‘ usw. bezeichnet.

Der Abschnitt zur Erzeugung eines Streckennetzes besteht aus zwei Bestandteilen bzw. Schritten. Zuerst werden Instanzen der Streckenstücke erstellt und im Anschluss werden die Instanzen zu einem Streckennetz verknüpft. Hierbei werden auch die Verbindungspunkte („Ports“), an denen das erzeugte Modul später mit anderen Modulen verknüpft werden kann, definiert.

Zunächst erfolgt die Instanziierung der Streckenstücke, die Sie im vorherigen Abschnitt ‚Courses‘ definiert haben oder aus SILABAEEdit in die Konfigurationsdatei über include-Anweisungen eingebunden haben. Dieses Prinzip kennen Sie bereits aus der DPU-Konfiguration, in der Instanzen von DPUs erstellt werden. Unter Instanziierung versteht man in der Informatik das Erzeugen eines Objekts (Streckenstück-Instanz) aus einer bestimmten Klasse (Streckenstück). Das bedeutet, dass in den Abschnitten unter ‚define Course Name‘ genau genommen erstmal nur Klassen (oder Typen) von Streckenstücken erstellt wurden. Nur die Instanzen eines Streckenstücks können letztlich befahren und miteinander zu einem Streckennetz verbunden werden. Die Instanziierung von Streckenstücken bietet den Vorteil, dass Sie mehrere Instanzen von einem Streckenstück erstellen und es somit mehrfach in ein Streckennetz einbauen können.

Bedingte Verknüpfungen: Durch die dynamische Datenbasis in SILAB besteht die Möglichkeit, die Verknüpfung von Streckenstücken zu einem Streckennetz an definierte Bedingungen zu knüpfen. Das Vorgehen zur Erstellung sogenannter bedingter Verknüpfungen wird in Kapitel 8 für die Verknüpfung von Szenario-Modulen zu einer Karte detailliert erklärt. Das gleiche Prinzip kann ebenfalls für die Erzeugung eines Streckennetzes aus Streckenstücken genutzt werden.

In dem darauffolgenden Abschnitt, der mit `Connections = { ... }`; eingeleitet wird, werden die Streckenstückinstanzen zu einem Straßennetz verknüpft.

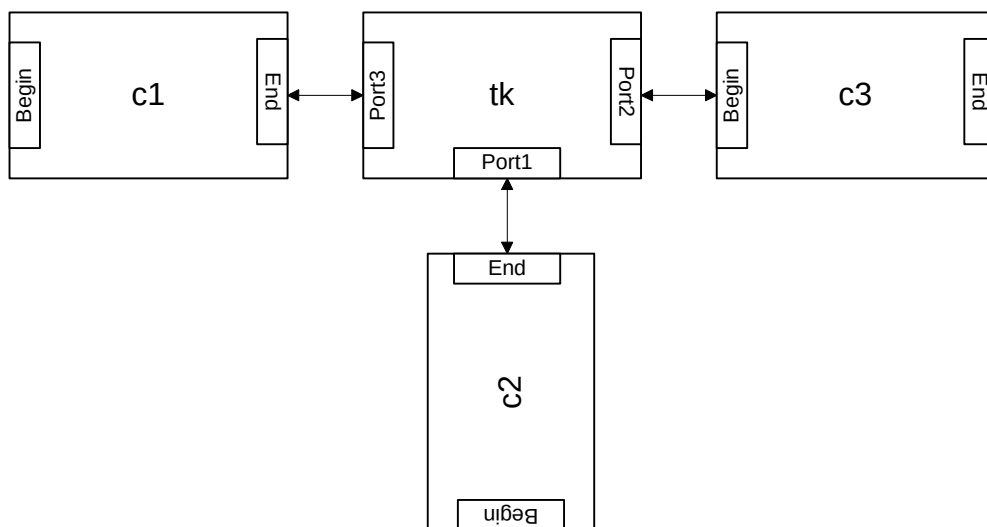


Abbildung 5. Streckennetz aus drei Courses und einer Area.

Im Folgenden wird nach der Vorlage in Abbildung 5 ein Streckennetz aus drei Courses mit den Namen ,C1', ,C2' und ,C3' und einer Area als T-Kreuzung mit dem Namen ,TKreuzung' erstellt. Die dazugehörigen Anweisungen in der Konfigurationsdatei sehen bspw. folgendermaßen aus:

```
### Erzeugung des Streckennetzes

### Instanziierung der Streckenstücke
C1 c1;
C2 c2;
C3 c3;
TKreuzung tk;
### /Instanziierung der Streckenstücke

### Verknüpfung der Instanzen zu einem Streckennetz
Connections =
{
c1.End    <-> tk.Port3,
c3.Begin <-> tk.Port2,
tk.Port1 <-> c2.End,
```

```
### Definition der Ports zur Verbindung mit anderen Modulen
c1.Begin <-> Port1,
c2.Begin <-> Port2,
c3.End   <-> Port3
### /Definition der Ports zur Verbindung mit anderen Modulen

};
### /Verknüpfung der Instanzen zu einem Streckennetz

### /Erzeugung des Streckennetzes
```

Code 16. Beispiel für die Erzeugung eines Streckennetzes mit den Unterebenen Instanziierung und Verknüpfung.

Die Anweisung zur Instanziierung der Streckenstücke hat diese Form:

```
<Name Streckenstück> <Name Streckenstückinstanz>
```

Die Namen für die Instanzen können Sie beliebig wählen. Eine übliche Konvention ist hierbei die Verwendung von Großschreibung für das Streckenstück und Kleinschreibung für Instanzen des Streckenstücks.

Der darauffolgende Abschnitt, in dem die Streckenstückinstanzen miteinander verknüpft werden, wird mit `Connections = { ... };` eingeleitet. Jedes Element entspricht einer Verknüpfung und hat diese Form:

```
<Name Streckenstückinstanz>.<Port> <-> <Name Streckenstückinstanz>.<Port>
```

In welcher Reihenfolge die Streckenstücke verknüpft werden und welcher Eintrag bei einer solchen Verknüpfung links oder rechts steht, ist egal, d.h. `c1.End <-> tk.Port3` ist gleichbedeutend mit `tk.Port3 <-> c1.End`.

Zur Verwendung des definierten Moduls in der Simulation müssen zusätzlich die Punkte, an denen das Modul mit anderen Modulen verknüpft werden kann, definiert werden. Die Anweisungen sind ähnlich zur Verknüpfung mit anderen Streckenstücken und haben folgende Form:

```
<Name Streckenstück-instanz>.<Port> <-> <Portname>
```

Portname ist hierbei definiert als der Text ‚Port‘, an den zusätzlich noch die Nummer des Ports angefügt ist (z.B. Port1).

Sollte das Streckennetz eines Szenariomoduls aus einer einzigen Area bestehen, wie z.B. ein Stadtteil, bietet SILABAEEdit die Möglichkeit, eine vollständige Moduldefinition zu erzeugen. Diese Moduldefinition enthält automatisch als einzige Streckenstückinstanz den entworfenen Stadtteil. Anstatt in der Konfigurationsdatei also das Modul mittels ‚define Module‘ zu definieren, können Sie die von SILABAEEdit erzeugte Moduldefinition mittels einer include-Funktion in die Konfigurationsdatei einbinden.

7 VERKEHRSSIMULATION FÜR EIN STRECKENNETZ

Die Verkehrssimulation von SILAB wird TRFX genannt und ermöglicht das Einfügen von einzelnen Verkehrsteilnehmern (Fahrzeuge, Motorräder und Fahrräder) oder ganzen Verkehrsflüssen in das Streckennetz, deren Verhalten vom Verhalten des Testfahrers oder vom Verhalten anderer Verkehrsteilnehmer gesteuert werden kann. Sie können Messwerte (Position, Geschwindigkeit, Beschleunigung usw.) jedes einzelnen Verkehrsteilnehmers aufzeichnen oder zur weiteren Verwendung mit anderen DPUs verknüpfen.

Die simulierten Verkehrsteilnehmer von TRFX fahren auf den Streckennetzen, die in den vorausgehenden Abschnitten der Moduldefinition mit der SILAB Datenbasis SCN/SCNX festgelegt wurden. Sie nutzen u.a. die festgelegten Spurinformatoren und Verkehrsregeln, um sich auf den Straßen zu bewegen.

7.1 Aufbau der Verkehrsdefinition in der Konfigurationsdatei

7.1.1 Allgemeine Angaben zur Verkehrssimulation

Um anderen Verkehr in der Simulation zu definieren, müssen in der Konfigurationsdatei im Abschnitt zu den allgemeinen Angaben zur Verkehrssimulation Einstellungen vorgenommen werden, die den gesamten Verkehr in allen Modultypen der Konfigurationsdatei betreffen. Dieser Abschnitt enthält z.B. die Zusammenfassung von einzelnen Verhaltensmodellen zu Verhaltensschemata, die Definition von Schaltprogrammen für Ampelanlagen und eine Tabelle mit Fahrzeugtypen und Gruppen von Fahrzeugen, die in der Konfigurationsdatei gebraucht werden. Der Lieferumfang von SILAB enthält die Datei SILAB_TRFX_700.cfg, in der sich die Standardeinstellungen für den Abschnitt zu den allgemeinen Angaben zur Verkehrssimulation befinden. Definieren Sie hierzu folgende Angaben im Skript:

```
SILAB TRF
{
include SILAB_TRFX_700.cfg

    # Zusätzliche Definitionen werden hier eingefügt
};
```

Code 17. Einbinden der Standardkonfiguration für TRFX in den Abschnitt mit den allgemeinen Angaben der Verkehrssimulation.

Normalerweise müssen Sie in diesem Abschnitt keine Ergänzungen vornehmen. Bei Bedarf finden Sie weitere Informationen zum Einfügen zusätzlicher Definitionen in den ‚SILAB TRF‘-Abschnitt in Kapitel 2.2 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).

7.1.2 Verkehrsdefinition für ein Streckennetz

Die Definition des Verkehrs für ein Streckennetz erfolgt in einem gesonderten Abschnitt innerhalb der Moduldefinition. Der Beginn dieses Abschnitts wird mit der Anweisung `Traffic` gekennzeichnet. Da sich alle simulierten Verkehrsteilnehmer im vorher definierten Streckennetz bewegen sollen, befindet sich dieser Abschnitt im Skript auf der gleichen Ebene wie der Abschnitt ‚Erzeugung eines Streckennetzes‘ (und nicht etwa innerhalb eines Abschnitts der Streckenstückdefinition). Code 18 zeigt den Aufbau der Verkehrsdefinition als Unterabschnitt der Szenariomoduldefinition. In diesem Beispiel besteht der Verkehr aus einem einzelnen Fahrzeug und einem Verkehrsfluss als Verkehr in Fahrtrichtung. Den genauen Aufbau der Definition einzelner Verkehrsteilnehmer und von Verkehrsflüssen erfahren Sie in den folgenden Abschnitten dieses Kapitels.

```
### Szenario-Modul

### Definition der Streckenstücke
### /Definition der Streckenstücke

### Erzeugung des Streckennetzes
### /Erzeugung des Streckennetzes

### Verkehrsdefinition
Traffic Name
{
  ### Einzelner Verkehrsteilnehmer
  VehicleX Name1
  {
    ...
  };
  ### /Einzelner Verkehrsteilnehmer

  ### Verkehrsfluss
  TrafficFlow Name2
  {
    ...
  };
  ### /Verkehrsfluss
};
### /Verkehrsdefinition

### /Szenario-Modul
```

Code 18. Aufbau der Verkehrsdefinition als Unterabschnitt der Szenario-Modul-Definition.

Für Area-Streckenstücke können Verkehrsteilnehmer und die Verkehrssteuerung mit dem Programm SILABAEEdit definiert werden. Wenn Sie ein komplettes Szenario-Modul mit Verkehrsdefinition in SILABAEEdit zusammengestellt haben und anschließend als Modul exportieren, wird die Verkehrsdefinition in eine separate include-Datei herausgeschrieben. Diese wird automatisch in das Modul eingebunden, so dass nur eine einzelne include-Anweisung erforderlich ist (s. Kapitel 5.2.1).

7.2 Verhaltensmodelle und Verhaltensschemata

Das Verhalten der simulierten Verkehrsteilnehmer wird über sogenannte Verhaltensmodelle gesteuert, die ihnen in der Verkehrsdefinition in der Konfigurationsdatei zugewiesen werden. Die hierarchische Kombination aus verschiedenen Verhaltensmodellen wird Verhaltensschema genannt. Diese werden in SILAB vordefiniert und sind in der Standardkonfiguration der Verkehrssimulation enthalten (SILAB_TRFX_700.cfg). Im Skript werden Verhaltensmodelle als ‚Behaviour‘ und Verhaltensschemata als ‚BehaviourScheme‘ bezeichnet.

Normalerweise werden Verhaltensmodelle in den gleichen hierarchischen Kombinationen eingesetzt. Aufgrund dessen werden in der Konfiguration der Verkehrssimulation größtenteils Verhaltensschemata anstatt einzelner Verhaltensmodelle definiert. Verhaltensschemata enthalten bereits Voreinstellungen von Parametern der Verhaltensmodelle. Diese können Sie selbstverständlich bei Bedarf verändern. Ein häufig verwendetes Verhaltensschema nennt sich ‚Standard‘. Es nutzt das Verhaltensmodell ‚EXSTD1‘ und stellt eine Kombination aus freier Fahrt und der Vermeidung von Kollisionen dar. Es generiert ein bestimmtes Geschwindigkeitsprofil für die/den Verkehrsteilnehmer. Um Auffahrunfälle zu vermeiden, wird der Verkehrsteilnehmer abgebremst, sobald ein bestimmter Sekundenabstand zum vorausfahrenden Verkehrsteilnehmer unterschritten wird.

Einen vollständigen Überblick über alle verfügbaren Verhaltensmodelle (Modellname, Verwendungszweck, Einschränkungen) finden Sie in Kapitel 5.3.1 im **DOKUMENT REFERENZ VERKEHRSSIMULATION TRFX**.

Kapitel 5.1.1 im Dokument **REFERENZ VERKEHRSSIMULATION TRFX** enthält einen Überblick über die Verhaltensschemata, die in den allgemeinen Angaben zur Verkehrssimulation (SILAB_TRFX_700.cfg) vordefiniert sind.

In den Abschnitten 7.4 und 7.5 lernen Sie, wie die Definition von Verhaltensschemata in der Verkehrsdefinition einzelner Fahrzeuge und von Verkehrsflüssen in der Konfigurationsdatei angegeben wird.

7.3 Verhaltenssteuerung durch Flowpoints

Während Verhaltensmodelle und -schemata definieren, wie sich andere Verkehrsteilnehmer verhalten sollen, muss in der Verkehrsdefinition darüber hinaus festgelegt werden, durch welchen Verkehrsteilnehmer, an welcher Stelle des Streckennetzes und/oder unter welchen Bedingungen das bestimmte Verhalten ausgelöst werden soll. Für jeden einzelnen Verkehrsteilnehmer und für jeden Verkehrsfluss, der in ein Szenariomodul hinzugefügt werden soll, muss definiert werden, wann und wie er aktiviert (und ggf. wieder deaktiviert) werden soll. Für diese Aufgaben sind in der Verkehrsdefinition sogenannte ‚Flowpoints‘ zuständig. Flowpoints haben eine ähnliche Funktion wie Stre-

cken-Events (s. Abschnitt 0) und enthalten eine Definition darüber, wann und wie Verhaltensmodelle bzw. Verhaltensschemata von Verkehrsteilnehmern ausgelöst werden. Damit legen Sie eine Art Drehbuch für die Verkehrssituation fest. Es wird zwischen streckenbezogenen und bedingten Flowpoints unterschieden.

Streckenbezogene Flowpoints sind unsichtbare Punkte auf Fahrspuren, bei deren Überfahren Parameter und Unterprogramme der Verhaltensmodelle verändert werden. Ein solcher Flowpoint wird ausgelöst, wenn die Stelle im Streckennetz, auf der dieser positioniert ist, überfahren wird. Die Definition der Stelle enthält Angaben über den Namen der entsprechenden Streckenstückinstanz, die Position des Flowpoints in Metern seit dem Start der Streckenstückinstanz und die Nummer der Spur (so wie sie in der Definition des Querschnittsprofils nummeriert wurde, s. Abschnitt 5.1.1), auf der der Flowpoint positioniert wird. Wahlweise können Flowpoints vom EGO-Fahrzeug oder von anderen Verkehrsteilnehmern überfahren und ausgelöst werden (s.u. durch Angabe des <Kontext> im Skript). Es kann zusätzlich definiert werden, dass erst beim Eintreten einer zusätzlichen Bedingung ein bestimmtes Verhalten ausgelöst wird, z.B. mit einer bestimmten zeitlichen Verzögerung nach Überfahren des Flowpoints (s.u. durch Angabe der <condition> im Skript). Kapitel 2.4.1.1 und 2.4.1.2 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#) enthalten eine vollständige Übersicht über die Namen und die Bedeutungen der unterschiedlichen Kontext- (engl. context) und Bedingungsangaben (engl. condition) der streckenbezogenen Flowpoints.

Ein bedingter Flowpoint wird über das Festlegen einer Bedingung (im Skript ‚condition‘ genannt) und einem zugehörigen Parameter definiert, z.B. sobald ein Verkehrsteilnehmer eine Strecke einer bestimmten Distanz zurückgelegt hat („DriveDistance“) oder nach Ablauf einer bestimmten Zeit, nachdem das EGO-Fahrzeug in ein Szenariomodul eingefahren ist („ModuleTime“). Eine vollständige Übersicht über die Namen und die zu definierenden Parameter der bedingten Flowpoints finden Sie in Kapitel 2.4.1.2 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).

In den folgenden Abschnitten 7.4 und 7.5 lernen Sie, wie Flowpoints im Rahmen der Verkehrsdefinition einzelner Fahrzeuge und von Verkehrsflüssen in der Konfigurationsdatei definiert werden.

7.4 Definition einzelner Verkehrsteilnehmer

In diesem Abschnitt erfahren Sie, wie Sie das Verhalten eines einzelnen Verkehrsteilnehmers auf einem Course in der Konfigurationsdatei definieren. Als Verkehrsteilnehmer liefert SILAB eine große Auswahl aus unterschiedlichen PKW- und LKW-Modellen, Bussen, Landmaschinen, Motorrädern und Fahrrädern. Eine Übersicht über die verschiedenen zur Auswahl stehenden Verkehrsteilnehmer finden Sie in Kapitel 5.1.4 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).

Im Folgenden finden Sie ein Beispiel für die Definition von zwei unterschiedlichen einzelnen Fahrzeugen in der Konfigurationsdatei.

```
### Verkehrsdefinition
Traffic MyTraffic
{
### Einzelnes Fahrzeug (Bsp. 1)
```

```

VehicleX Vorderfahrzeug
{
    Vehicle = OpelZafira;
    Colour = white;
    Position = (c1, 50.0, 5);
    BehaviourScheme = Standard;
    StartSpeed = 0.0;
    TargetSpeed = 0.0;
    Flowpoints =
    {
        (ModuleTime, 0, Activate),
        (c1, 1, 6, SimCar, TargetSpeedAndAcceleration, 27.78, 1.0)
    };
};
### /Einzelnes Fahrzeug (Bsp. 1)

### Einzelnes Fahrzeug (Bsp. 2)
VehicleX Fahrzeug2
{
    Vehicle = Car;
UserID = 1;
    Position = (c1, 500.0, 4);
    BehaviourScheme = Standard;
    Flowpoints =
    {
        (c1, 100, 5, SimCar, Activate),
        (c1, 600, 4, Traffic, TargetSpeed, 0),
        (c1, 900, 5, SimCar, Deactivate)
    };
};
### /Einzelnes Fahrzeug (Bsp. 2)

};
### /Verkehrsdefinition

```

Code 19. Beispiel für die Konfiguration von zwei einzelnen Verkehrsteilnehmern.

Für Areas erfolgt die Definition einzelner anderer Verkehrsteilnehmer mit dem grafischen Editor SILABAEEdit. Dieses Vorgehen wird in Kapitel 3.16 im *Handbuch SILABAEEdit* beschrieben und wird an dieser Stelle nicht weiter thematisiert. Die wesentlichen Konzepte (z.B. streckenabhängige vs. bedingte Flowpoints), die hier für Course-Streckenstücke erläutert werden, treffen auch für Areas zu.

Damit Sie die einzelnen Anweisungen, die in der Definition der beiden einzelnen Fahrzeuge in Code 19 enthalten sind, besser nachvollziehen können, folgt nun eine schrittweise Erläuterung der Syntax:

Die Verkehrsdefinition wird mit dem Schlüsselwort **Traffic** eingeleitet.

Traffic <Name>

<Name> Der Verkehrsdefinition für das Szenariomodul wird ein möglichst aussagekräftiger Name gegeben, den Sie frei wählen dürfen (in Code 19: MyTraffic)

Die einzelnen Fahrzeuge werden innerhalb einer großen Klammer auf einer Unterebene der Verkehrsdefinition nacheinander definiert. Die Definition einzelner Verkehrsteilnehmer wird jeweils mit dem Schlüsselwort `VehicleX` eingeleitet.

VehicleX <Name>

<Name> Dem Verkehrsteilnehmer wird ein möglichst aussagekräftiger Name gegeben, den Sie frei wählen dürfen (in Code 19: Vorderfahrzeug und Fahrzeug2)

Die Definition eines einzelnen Verkehrsteilnehmers hat verschiedene Bestandteile, für die jeweils Parameter definiert werden:

Einem Verkehrsteilnehmer kann eines von zahlreichen Fahrzeugen zugewiesen werden. Im Lieferumfang von SILAB sind als Fahrzeuge verschiedene Modelle von PKW, Kleintransporter und LKW in mehreren Farben, sowie Motorräder und Fahrräder enthalten.

Vehicle = <Fahrzeugtyp>;

<Fahrzeugtyp> Festlegen des Fahrzeugtyps für den Verkehrsteilnehmer. Sie können als <Fahrzeugtyp> hier spezielle Fahrzeugmodelle auswählen (Bsp. 1: Vehicle = OpelZafira) oder einen Gruppennamen für eine Fahrzeugart eingeben, woraufhin zufällig ein Fahrzeug aus dieser Gruppe eingefügt wird (Bsp. 2: Vehicle = Car). Alle möglichen Bezeichnungen für <Fahrzeugtyp> finden Sie in der ‚RoadUserTable‘ in Kapitel 5.1.4 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).

Optional können Sie dem Verkehrsteilnehmer eine (eindeutige) Kennnummer zuweisen. Über diese UserID können Sie sein Verhalten an anderen Stellen der Verkehrsdefinition über öffentliche Flowpoints im Rahmen der Kontextdefinition steuern (z.B. über SimCarUserID oder TrafficUserID).

UserID = <Index>;

Optional kann die Farbe des Fahrzeugs angegeben werden.

Colour = <Farbname>;

<Farbname> Eine Liste der vordefinierten Farben in SILAB finden Sie in der ‚RoadUserTable‘ in Kapitel 5.1.4 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).

Die anfängliche Streckenposition des Fahrzeugs wird festgelegt:

Position = (<Streckenstück-Instanz>, <Streckenmeter>, <Spurnummer>);

<Streckenstück-Instanz> Der Name der Streckenstück-Instanz, auf dem das Fahrzeug initial positioniert werden soll

<Streckenmeter> legt die Position des Fahrzeugs entlang der Straße fest, gemessen in Metern vom Beginn der Streckenstück-Instanz aus.

<Spurnummer> Die Nummer der Spur, auf der das Fahrzeug positioniert werden soll (die Spurnummern werden im Querschnittsprofil unter LaneInfos festgelegt, s. Abschnitt 5.1.1)

Zur Definition des Verhaltens des Fahrzeugs wird entweder ein Verhaltensschema (BehaviourScheme) oder eine Menge an Verhaltensmodellen (Behaviour) angegeben. Die Form zum Einfügen eines Verhaltensschemas sieht folgendermaßen aus:

BehaviourScheme = <Name>;

<Name> Hier tragen Sie den Namen des gewünschten Verhaltensschemas ein. Kapitel 5.1.1 im Dokument **REFERENZ VERKEHRSSIMULATION TRFX** enthält einen Überblick über die vordefinierten Verhaltensschemata.

Alternativ kann das Verhalten über eine Menge an Verhaltensmodellen definiert werden:

Behaviour = {<ModellX>, <ModellY>, ...};

<ModellX> Hier tragen Sie das Tupel zur Definition des gewünschten Verhaltensmodells ein. Kapitel 5.3.1 im Dokument enthält einen Überblick über die vordefinierten Verhaltensmodelle.

Alle Flowpoints werden als Elemente der Menge Flowpoints = {...}; aufgeführt. Jedes Element entspricht einem Flowpoint.

Streckenbezogene Flowpoints (s. Abschnitt 7.3) haben folgende Form:

(<Streckenstück-Instanz>, <Streckenmeter>, <Spurnummer>, <Kontext>, <Flowpoint-Informationen>)

<Streckenstück-Instanz>, <Streckenmeter>, <Spurnummer> s. Beschreibung der Parameter der Position

<Kontext> Die Angabe unter <Kontext> bestimmt, durch welchen Verkehrsteilnehmer in der Simulation der Flowpoint ausgelöst wird. Eine Liste der möglichen Angaben und deren Bedeutung finden Sie

in Kapitel 2.4.1.1 im Dokument **REFERENZ VERKEHRSSIMULATION TRFX**.

<Flowpoint-Informationen>

<Flowpoint-Informationen> beschreiben, was beim Überfahren des Flowpoints passieren soll. Je nach Flowpoint-Information wird hier ein Parameter oder mehrere Parameter ergänzt, z.B. die Zielgeschwindigkeit in m/s für den Eintrag `TargetSpeed` (s. 2. Flowpoint in Bsp. 2 von Code 19). Eine Übersicht finden Sie in den Kapiteln 5.3.5 bis 5.3.7 jeweils unter ‚Flowpoints‘ im Dokument **REFERENZ VERKEHRSSIMULATION TRFX**.

Bedingte Flowpoints (s. Abschnitt 7.3) haben folgende Form:

(<Bedingung>, <Wert>, <Flowpoint-Informationen>)

<Bedingung>,
<Wert>

Ein bedingter Flowpoint wird über die Festlegung einer <Bedingung> und zugehöriger Werte definiert. Je nach Art der Bedingung werden ein oder zwei Werte angegeben. Bspw. gibt die Bedingung `ModuleTime` an, dass der Flowpoint nach Ablauf von <Wert> Sekunden seit der Einfahrt des EGO-Fahrzeugs in das Szenariomodul ausgelöst wird. Eine Liste der möglichen Bedingungen und dazugehörigen Wertangaben finden Sie in Kapitel 2.4.1.1 im Dokument **REFERENZ VERKEHRSSIMULATION TRFX**.

<Flowpoint-Informationen>

s. Beschreibung für streckenbezogene Flowpoints

Mit diesen Informationen zum Inhalt der Definition einzelner Fahrzeuge in einem Szenario-Modul werden nun die beiden Fahrzeuge aus Code 19 erläutert:

In Beispiel 1 wird ein Fahrzeug des Modells Opel Zafira in weißer Farbe eingefügt, der als Vorderfahrzeug dienen soll. Das Fahrzeug wird auf dem Streckenstück `c1` in einer Entfernung von 50 m entlang der Straße auf der Spur 5 erzeugt (`Position = (c1, 50.0, 5)`). Als Verhaltensschema wird das Standardverhalten ausgewählt, welches eine Kombination aus freier Fahrt und der Vermeidung von Kollisionen mit anderen Fahrzeugen darstellt. Da das Fahrzeug zunächst auf den Fahrer der Simulation warten soll, werden sowohl die Start- als auch die Zielgeschwindigkeit zunächst auf 0 m/s gesetzt. Der erste (bedingte) Flowpoint (`ModuleTime, 0, Activate`) wird ausgelöst, wenn eine Zeit von 0 Sekunden seit der Aktivierung des Szenarios vergangen ist, also unmittelbar beim Start der Simulation. Die Flowpoint-Information `Activate` bedeutet, dass das vorausfahrende Fahrzeug zu diesem Zeitpunkt erzeugt wird. Der zweite (streckenbezogene) Flowpoint liegt auf dem Streckenstück `c1`, auf Streckenmeter 1 und auf Spur 6 (das EGO-Fahrzeug befindet sich zu Beginn der Simulation auf Spur 6 (Standstreifen) und wechselt dann auf Spur 5). Der Kontext `SimCar` bedeutet, dass der Flowpoint vom Fahrer ausgelöst wird. Die Flowpoint-Information `TargetSpeedandAcceleration` hat zwei Parameter und bedeutet, dass die Zielgeschwindigkeit bei 27.87 m/s (= 100 km/h) liegt und mit einer Beschleunigung von 1 m/s² erreicht

wird. Das EGO-Fahrzeug würde diesen zweiten Flowpoint also direkt beim Losfahren überfahren und somit das andere Fahrzeug zum Losfahren auf Spur 5, 50 m vor ihm, bewegen. Nachdem das EGO-Fahrzeug ein paar Sekunden auf Spur 5 gefahren ist, wird der Opel Zafira vor ihm erscheinen und als Vorderfahrzeug dienen.

In Beispiel 2 wird zufällig ein Fahrzeug der Gruppe Car eingefügt. Das Fahrzeug wird auf dem Streckenstück c1 in einer Entfernung von 500 m entlang der Straße auf Spur 4 erzeugt (`Position = (c1, 500.0, 4)`). Das Fahrzeug verhält sich entsprechend dem Verhaltensschema Standard. Die drei Flowpoints geben an, dass das Fahrzeug vom EGO-Fahrzeug (SimCar) erzeugt wird, sobald dieses den 100. Streckenmeter auf Spur 5 überfährt. Wenn das erzeugte Fahrzeug den 600. Streckenmeter auf Spur 4 überfährt, bremst es in den Stillstand (`TargetSpeed, 0`). Sobald das Ego-Fahrzeug den 900. Streckenmeter auf Spur 5 passiert hat (also schon am eingefügten Fahrzeug vorbeigefahren ist), wird das andere Fahrzeug wieder deaktiviert.

Wenn Sie ein eingefügtes Fahrzeug nicht manuell durch einen Flowpoint wieder deaktivieren, wird dieses Fahrzeug entfernt, sobald das Ego-Fahrzeug das Szenario-Modul verlässt.

In Kapitel 2.7 im Dokument *Referenz Verkehrssimulation TRFX* finden Sie Beispiele für unterschiedliche Anwendungsfälle der Konfiguration einzelner Verkehrsteilnehmer, z.B. für ein plötzliches Bremsen eines vorausfahrenden Fahrzeugs oder für ein vorausfahrendes und abbiegendes Fahrzeug.

7.5 Definition eines Verkehrsflusses

In diesem Abschnitt lernen Sie, wie Sie innerhalb der Verkehrsdefinition einen ganzen Verkehrsfluss (engl. ‚Traffic Flow‘) steuern können. Der Gegenverkehr auf einer Landstraße oder einer Autobahn ist ein typisches Beispiel für die Verwendung eines Verkehrsflusses auf einem Course-Streckenstück. In diesem Fall ist das Verhalten eines einzelnen Fahrzeugs des Gegenverkehrs nicht von Bedeutung. Es reicht, dem Fahrer eine Menge von Fahrzeugen mit gleicher Geschwindigkeit in variierenden Abständen entgegenkommen zu lassen.

Ein Verkehrsfluss nutzt sogenannte ‚Quellen‘ (Vgl. `Source` in `Code 20`), um Verkehrsteilnehmer nach bestimmten Regeln im Streckennetz zu platzieren und ‚Senken‘ (Vgl. `Drain` in `Code 20`), um die Verkehrsteilnehmer wieder zu deaktivieren bzw. verschwinden zu lassen. Quellen und Senken können sowohl statisch auf einer bestimmten Position platziert, als auch mit dem EGO-Fahrzeug bewegt werden. Im zweiten Fall wird die Quelle sozusagen an das EGO-Fahrzeug geheftet und fährt diesem bspw. in einer konstanten Entfernung von 500 m auf der Gegenseite voraus. Eine Quelle enthält eine definierte Anzahl an Fahrzeugen. An ihrer jeweiligen Position setzt die Quelle die Verkehrsteilnehmer in definierten Abständen aus ihrer Menge auf die Gegenseite, solange noch inaktive Verkehrsteilnehmer in ihrer Menge enthalten sind. Sind alle Verkehrsteilnehmer der Quelle zum aktuellen Zeitpunkt aktiviert, unterbricht sie ihren Verkehrsfluss, bis sich wieder inaktive Verkehrsteilnehmer in ihrer Menge befinden.

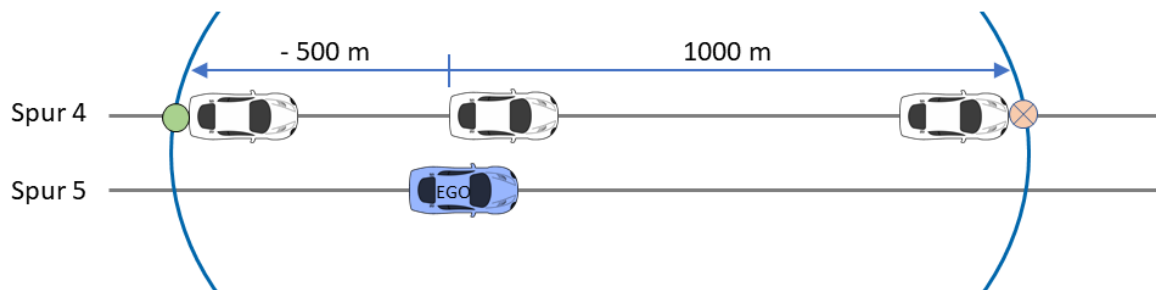


Abbildung 6. Visualisierung eines Verkehrsflusses, der an das EGO-Fahrzeug angeheftet ist und mit diesem mitfährt. Der grüne Kreis stellt die Quelle und der rote Kreis die Senke dar.

Im Folgenden finden Sie ein Beispiel für die Definition zweier Verkehrsflüsse in der Konfigurationsdatei. Diese werden als Gegenverkehr bzw. als Mitverkehr in Fahrtrichtung des EGO-Fahrzeugs eingesetzt. Die Situation wird in Abbildung 6 visualisiert.

```
### Verkehrsdefinition
Traffic MyTraffic
{
  ### Verkehrsfluss 1: Mitverkehr in Fahrtrichtung
  TrafficFlow Mitverkehr
  {
    Source Q_Mitverkehr
    {
      Position = (SimCar, -500, 4);
      Vehicles =
```

```

{
(15, Car, (HazardFree, 35, 35, 5, 1, 10, -14)),
(3, Truck, (HazardFree, 22, 22, 5, 2, 10, -14))
};
Parameters = (Gauss, 3.0, 1.4, 451, 512);
};

Drain S_Mitverkehr
{
Position = (SimCar, 1000, 4);
};

Flowpoints =
{
(c1, 500, 5, SimCar, ActivateSource, (Q_Mitverkehr)),
(c1, 500, 5, SimCar, ActivateDrain, (S_Mitverkehr))
};
};
### /Verkehrsfluss 1: Mitverkehr in Fahrtrichtung

### Verkehrsfluss 2: Gegenverkehr
TrafficFlow Gegenverkehr
{
Source Q_Gegenverkehr
{
Position = (SimCar, -900, 1);
Vehicles =
{
(10, Car, (HazardFree, 22, 22, 5, 2, 10, -14))
};
Rotate = 1;
Parameters = (Uniform, 1, 4, 451, 512);
};

Drain S_Gegenverkehr
{
Position = (SimCar, 920, 1);
};

Flowpoints =
{
(c1, 500, 5, SimCar, ActivateSource, (Q_Gegenverkehr)),
(c1, 500, 5, SimCar, ActivateDrain, (S_Gegenverkehr))
};
};
### /Verkehrsfluss 2: Gegenverkehr
};
### /Verkehrsdefinition

```

Code 20. Beispiel für die Konfiguration von zwei Verkehrsflüssen.

Die Definition eines Verkehrsflusses besteht aus einem ersten Abschnitt, in dem die Quelle definiert wird, einem zweiten Abschnitt, in dem die Senke definiert wird, und einem dritten Abschnitt mit

der Definition der Flowpoints. Im Folgenden erhalten Sie genauere Informationen über die Syntax zur Definition eines Verkehrsflusses im Rahmen der Verkehrsdefinition.

Die verschiedenen Verkehrsflüsse werden (so wie die Definition einzelner Verkehrsteilnehmer auch) innerhalb einer großen Klammer auf einer Unterebene der Verkehrsdefinition nacheinander definiert. Die Definition eines Verkehrsflusses wird jeweils mit dem Schlüsselwort `TrafficFlow` eingeleitet.

`TrafficFlow <Name>`

`<Name>` Dem Verkehrsfluss wird ein möglichst aussagekräftiger Name gegeben, den Sie frei wählen können (z.B. Mitverkehr)

Die Definition der Quelle wird mit dem Schlüsselwort `Source` eingeleitet. Innerhalb dieser Definition werden die Position der Quelle, darin enthaltene Verkehrsteilnehmer und deren zeitliche Folge festgelegt.

`Source <Name>`

`<Name>` Der Quelle des Verkehrsflusses wird ein möglichst aussagekräftiger Name gegeben, den Sie frei wählen können (im Beispiel: Q_Mitverkehr und Q_Gegenverkehr)

Die Position der Quelle wird in Form von einer der beiden folgenden Alternativen definiert. Im ersten Fall wird die Quelle ortsfest in der Datenbasis positioniert und im zweiten Fall wird die Quelle an das EGO-Fahrzeug geheftet und fährt mit dem EGO-Fahrzeug mit (s. Abbildung 6 zur Visualisierung).

`Position = (<Streckenstück-Instanz>, <Streckenmeter>, <Spurnummer>);`
`Position = (SimCar, <Abstand>, <Spurnummer>);`

`<Streckenstück-Instanz>` Der Name der Streckenstück-Instanz, auf dem die Quelle positioniert werden soll

`<Streckenmeter>` legt die Position der Quelle entlang der Straße fest, gemessen in Metern vom Beginn der Streckenstückinstanz aus.

`<Spurnummer>` Die Nummer der Spur, auf der die Quelle positioniert werden soll (die Spurnummern werden im Querschnittsprofil unter `LaneInfos` festgelegt, s. Abschnitt 5.1.1)

`SimCar, <Abstand>` Gibt den Abstand der Quelle zum EGO-Fahrzeug an. Bei Fahrspuren in Fahrtrichtung des EGO-Fahrzeugs liegen Abstände mit einem positiven Wert vor dem EGO-Fahrzeug und Abstände mit negativen Werten hinter dem EGO-Fahrzeug. Bei Fahrspuren entgegen der Fahrtrichtung des EGO-Fahrzeugs liegen Abstände mit einem positiven Wert hinter dem EGO-Fahrzeug und Abstände mit negativen Werten vor dem EGO-Fahrzeug.

Definition der Fahrzeugliste: Die Verkehrsteilnehmer, die in der Menge der Quelle enthalten sind, werden als Elemente der Menge Vehicles = {...} definiert. Bei jedem neu generierten Fahrzeug sucht SILAB nach einem noch nicht verwendeten Fahrzeug in dieser Fahrzeugliste. Jedes Element bezieht sich auf einen Fahrzeugtyp und hat diese Form:

(`<Anzahl>`, `<Fahrzeugtyp>`, (`<Verhaltensmodell>`))

`<Anzahl>` Anzahl der verschiedenen Fahrzeuge von dem ausgewählten Fahrzeugtyp

`<Fahrzeugtyp>` Eine Liste der verfügbaren Fahrzeugtypen finden Sie in der RoadUser-Table in Kapitel 5.1.4 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#).

(`<Verhaltensmodell>`) Es können mehrere Verhaltensmodelle für einen Fahrzeugtyp definiert werden. Die einzelnen Verhaltensmodelle werden per Komma voneinander getrennt. Pro Verhaltensmodell wird als erster Eintrag in der Klammer der Name des Verhaltensmodells angegeben. Es folgen Parameterangaben, die sich für verschiedene Verhaltensmodelle optional unterscheiden. Einen vollständigen Überblick über alle verfügbaren Verhaltensmodelle (Modellname, Verwendungszweck, Einschränkungen) finden Sie in Kapitel 5.3.1 im Dokument [REFERENZ VERKEHRSSIMULATION TRFX](#). Die Parameter finden Sie von Kapitel 5.3.3.1 bis 5.3.7.2 im gleichen Dokument.

Optional können Sie mit dem Parameter `Rotate` angeben, nach welchem Muster SILAB die Fahrzeuge aus der definierten Fahrzeugliste zieht. Der Wert 0 definiert, dass SILAB an der ersten Position nach einem noch nicht verwendeten Fahrzeug in der Fahrzeugliste sucht. Bei einer großen Menge an Fahrzeugen kann es daher vorkommen, dass die zuletzt definierten Fahrzeuge nie erscheinen. Wird der Wert auf 1 gesetzt, werden Fahrzeuge erst ein zweites Mal platziert, wenn bereits alle Quellenfahrzeuge einmal gezogen wurden.

`Rotate = 0|1;`

Für jede Quelle wird die zeitliche Folge der erzeugten Verkehrsteilnehmer festgelegt. Hierzu wird in jedem Simulationsschritt die Distanz in Sekunden zwischen dem zuletzt erzeugten Verkehrsteilnehmer und der Quelle ermittelt. Es können normalverteilte (`Gauss`) und gleichmäßige Abstände (`Uniform`) definiert werden.

`Parameter = (Gauss, <μ>, <σ>, <period>, <seed>);`

`Parameter = (Uniform, <Min-Wert>, <Max-Wert>, <period>, <seed>);`

`<μ>`, `<σ>` Mittelwert `<μ>` und Standardabweichung `<σ>` der Normalverteilung

`<period>`, `<seed>` Bei der Erzeugung der Zufallszahlen wird eine Zufallstabelle mit der angegebenen Anzahl von Zufallswerten `<period>` und dem angegebenen Samen `<seed>` für den Zufallsgenerator erzeugt. Ein Same ist eine

beliebige ganze Zahl. Ein bestimmter Samen erzeugt bei jedem Simulationsstart die gleiche Zufallsreihenfolge. Damit ist das Verhalten von Verkehrsflüssen zwar zufällig aber zwischen verschiedenen Simulatorfahrten (z.B. bei verschiedenen Durchläufen eines Experiments) reproduzierbar.

<Min-Wert>,
<Max-Wert>

Die Zufallsfolge ist gleichverteilt mit dem angegebenen Minimal- und Maximalwert.

Die Definition der Senke wird mit dem Schlüsselwort `Drain` eingeleitet. Innerhalb dieser Definition wird die Position der Senke festgelegt. Wird die Position relativ zum EGO-Fahrzeug (SimCar) angegeben, wird diese als Radius um das EGO-Fahrzeug interpretiert. Die Benennung der Senke sowie die Definition der Position erfolgen analog zur Quellendefinition.

Die Flowpoints zur Aktivierung und Deaktivierung der Quellen und Senken werden als Elemente der Menge `Flowpoints = {...}`; definiert. Die Definition der Flowpoints erfolgt wie in Abschnitt 7.4 beschrieben in Form streckenbezogener oder bedingter Flowpoints. Die Flowpoint-Informationen innerhalb der Flowpoint-Definition haben immer eine der folgenden Formen:

<code>ActivateSource,</code> <code>(<Name>)</code>	Aktivierung der angegebenen Quelle
<code>DeactivateSource,</code> <code>(<Name>)</code>	Deaktivierung der angegebenen Quelle
<code>ActivateDrain,</code> <code>(<Name>)</code>	Aktivierung der angegebenen Senke
<code>DeactivateDrain,</code> <code>(<Name>)</code>	Deaktivierung der angegebenen Senke

Bei der Platzierung von Senken zum Deaktivieren von Fahrzeugen relativ zum EGO-Fahrzeug wird die angegebene Distanz als Radius interpretiert. Daher ist es unbedingt erforderlich, die Distanz auf einen größeren Wert als die relative Position der Quelle zu setzen. Andernfalls werden die Fahrzeuge der Quelle platziert und direkt wieder durch die Senke gelöscht.

Mit diesen Informationen zum Inhalt der Definition eines Verkehrsflusses in einem Szenario-Modul wird nun der erste Verkehrsfluss (Mitverkehr) aus dem Beispiel-Skript erläutert:

Es wird ein Verkehrsfluss mit dem Namen ‚Mitverkehr‘ generiert. Abbildung 6 stellt die Definition dieses Verkehrsflusses grafisch dar. Die Quelle dieses Verkehrsflusses wird mit `Q_Mitverkehr` benannt und an das EGO-Fahrzeug geheftet. Die Position der Quelle befindet sich immer 500 m hinter dem EGO-Fahrzeug auf Spur 4. Es wird eine Fahrzeug-Liste definiert, die festlegt, dass die Quelle 15 PKWs zur Verfügung hat, die zufällig aus der Gruppe `Car` gezogen werden. Zusätzlich

werden zufällig 3 LKWs aus der Gruppe `Truck` gezogen. Alle Fahrzeuge der Quelle verhalten sich entsprechend der Verhaltensmodelle `HazardFree` zur Vermeidung von Auffahrunfällen und zum Fahren mit einer Geschwindigkeit von 35 m/s (= 126 km/h), bzw. 22 m/s (= 80 km/h) für LKW. Die Angabe für Parameter definiert, dass die Verkehrsteilnehmer mit normalverteilten zeitlichen Abständen erzeugt werden. Der Mittelwert der Abstände beträgt 3 Sekunden und die Standardabweichung 1.4 Sekunden. Die Senke mit dem Namen ‚S_Mitverkehr‘ befindet sich immer in einem Radius von 1000m um das EGO-Fahrzeug auf Spur 4. Es gibt zwei streckenbezogene Flowpoints, die auf derselben Position liegen (Streckenstück c1, Streckenmeter 500, Spur 5) und vom Fahrer ausgelöst werden (Kontext `SimCar`). Der erste Flowpoint aktiviert die Quelle, der zweite aktiviert die Senke.

7.6 Fußgängersimulation

Das SILAB-Modul `MOV` ermöglicht die Simulation von Fußgängern innerhalb von Stadtszenarien, d.h. Streckenstücke des Typs `Area`. Die Standardkonfiguration der Fußgängersimulation, die Fußgängermodelle und -animationen beinhaltet, sollte im Abschnitt ‚Allgemeine Angaben zu weiteren Komponenten der Simulation‘ der Konfigurationsdatei als include-Datei (`include MOV/MOP_700.inc`) eingefügt werden.

Die Fußgängersimulation kann ausschließlich in `Areas` erfolgen. Auf `Courses` ist die Fußgängersimulation nicht möglich. Dafür werden im grafischen Editor `SILABAEEdit` Fußgängerbereiche (d.h. Gehwege oder Plätze) definiert, auf denen sich die Fußgänger daraufhin frei bewegen. Die virtuellen Personen wählen eigenständig aus einer Menge vordefinierter Ziele und weichen auf dem Weg dorthin selbstständig Hindernissen, dem EGO-Fahrzeug und Fremdverkehr aus der Verkehrssimulation `TRFX` aus. Außerdem nutzen sie Fußgängerampeln und -überwege regelgemäß.

Des Weiteren besteht die Möglichkeit, einzelne Fußgänger gezielt zu steuern, um kritische Verkehrssituationen herzustellen. Beispielsweise kann mit `SILABAEEdit` ein Fußgänger so konfiguriert werden, dass er bei Annäherung des Testfahrers die Straße überquert und diesen so zum Handeln zwingt, um eine Kollision zu vermeiden.

Detaillierte Informationen zur Fußgängersimulation und deren Anwendung finden Sie im Dokument **REFERENZ FUßGÄNGERSIMULATION MOV** und in Kapitel 3.16 im **HANDBUCH SILABAEEDIT**.

8 ERZEUGUNG EINER KARTE AUS SZENARIOMODULEN

Nachdem Sie verschiedene Module erstellt haben, werden diese miteinander zu einer Karte verbunden. Diese Karte bildet die Einheit aller Bestandteile des Straßennetzwerkes, die Sie zuvor erstellt haben.

Der Abschnitt ‚Erzeugung der Karte aus den Szenario-Modulen‘ befindet sich in der Konfigurationsdatei auf der gleichen Ebene wie die Definition der einzelnen Module. Das Vorgehen zur Erzeugung einer Karte aus Szenariomodulen ist sehr ähnlich zu dem Vorgehen zur Erzeugung eines Straßennetzes aus Streckenstücken, welches in Kapitel 6 beschrieben wird.

Jedes Modul besitzt Anschlüsse, mit denen es mit anderen Modulen verbunden werden kann. Diese Anschlüsse werden ‚Port‘ genannt. Die Anzahl der Ports pro Area hängt von der Streckengeometrie ab. Eine klassische Kreuzung verfügt z.B. über vier und eine T-Kreuzung über drei Ports. SILABAEEdit vergibt die Namen der Ports automatisch, die immer mit ‚Port1‘, ‚Port2‘, ‚Port3‘ usw. bezeichnet werden. Wenn ein Modul aus Courses und Areas besteht, ergibt sich die Anzahl der Ports aus der Definition des Moduls und der Anzahl an Ports, die nicht mit anderen Streckenstücken des Moduls verbunden sind. Das Modul, das sich aus dem Streckennetz in Abbildung 5 ergibt, besitzt bspw. die Ports Port1, Port2 und Port3.

Der Abschnitt zur Erzeugung einer Karte besteht aus zwei Schritten. Als Erstes erfolgt die Instanziierung der Module, die Sie zuvor in der Konfigurationsdatei definiert haben oder aus SILABAEEdit in die Konfigurationsdatei über include-Anweisungen eingebunden haben. In Kapitel 6 finden Sie eine detaillierte Erklärung dieses Prinzips. Im zweiten Schritt werden diese Instanzen zu der Karte verknüpft. Dieser Abschnitt, in dem der Modul-Ablauf definiert wird, wird mit `Connections = { . . . }` eingeleitet. Darüber hinaus werden in einem weiteren Unterabschnitt die Aufsetzpunkte der Karte definiert. Auf dieses Vorgehen wird in Abschnitt 8.1 näher eingegangen.

Im Folgenden wird nach der Vorlage in Abbildung 7 eine Karte aus drei Modulen mit den Namen ‚ModulX‘, ‚ModulY‘ und ‚ModulZ‘ erstellt. Hier sehen Sie, wie die dazugehörigen Anweisungen in der Konfigurationsdatei aussehen würden:

```
### Erzeugung der Karte aus den Szenariomodulen

Map Kartenname
{
  ### Instanziierung der Module
  ModulX m1;
  ModulY m2;
  ModulZ m3;
  ### /Instanziierung der Module

  ### Verknüpfung der Instanzen zu einer Karte
  Connections =
  {
    m1.Port2 <-> m2.Port1,
    m2.Port2 <-> m3.Port1,
```

```

m2.Port3 <-> m3.Port1
};
### /Verknüpfung der Instanzen zu einer Karte

### Definition der Aufsetzpunkte
{
# s. Abschnitt 8.1
};
### /Definition der Aufsetzpunkte
};
### /Erzeugung der Karte aus den Szenariomodulen

```

Code 21. Beispiel für die Erzeugung der Karte aus Szenariomodulen.

Die Anweisung zur Instanziierung der Module hat diese Form:

```
<Name Modul> <Name Modulinstanz>;
```

Die Namen für die Instanzen können Sie beliebig wählen. An dieser Stelle werden auch Areas instanziiert, die Sie als ganze Szenariomodule aus SILABAEEdit exportiert haben.

Der darauffolgende Abschnitt, in dem die Modulinstanzen miteinander zu einer Karte verknüpft werden, wird mit `Connections = {...};` eingeleitet. Jedes Element entspricht einer Verknüpfung und hat diese Form:

```
<Name Modul-Instanz>.<Port> <-> <Name Modul-Instanz>.<Port>
```

In welcher Reihenfolge die Modulinstanzen verknüpft werden und welcher Eintrag bei einer solchen Verknüpfung links oder rechts steht, ist irrelevant, d.h. `m1.Port2 <-> m2.Port1` ist gleichbedeutend mit `m2.Port1 <-> m1.Port2`.

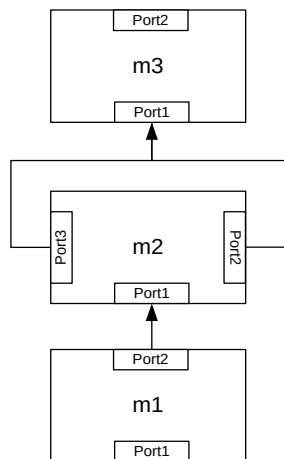


Abbildung 7. Visualisierung einer Karte aus drei Modul-Instanzen.

Bei der Erzeugung der Karte bietet die dynamische Datenbasis von SILAB Vorteile. Im Gegensatz zu festen Straßennetzwerken in der Realität muss in der Simulation nur das Straßennetzwerk im unmittelbaren Sichtbereich des Fahrers festgelegt werden. Außerhalb des Sichtbereichs kann das Straßennetzwerk sowie andere Verkehrsteilnehmer und die Umgebung beliebig verändert werden (s.

Kapitel 5.1 im **HANDBUCH SILAB ESSENTIAL** für eine detaillierte Erklärung). Somit können Sie durch die Verknüpfung von Modulen dafür sorgen, dass der Fahrer diese immer in der von Ihnen gewünschten Reihenfolge durchfährt. Sie haben die Möglichkeit, verschiedene Abbiegungen einer Kreuzung mit demselben darauffolgenden Modul zu verknüpfen. Wenn Sie beispielsweise möchten, dass der Fahrer die Instanzen m1, m2 und m3 von Szenariomodulen genau in dieser Reihenfolge durchfährt, können Sie die Instanzen, so wie in Abbildung 7 dargestellt, miteinander zu einer Karte verknüpfen. Durch diese Methodik wird die gewünschte Modulabfolge unabhängig von der Wegwahl des Fahrers sichergestellt. Wenn der Fahrer in der Modulinstanz m2 rechts abbiegt, wird Port2 sichtbar und m2 wird mit m3 verknüpft. Biegt er in m2 links ab, wird m3 an Port3 von m2 angeschlossen. Erst zu dem Zeitpunkt, an dem ein Port einer Modul-Instanz für den Fahrer sichtbar wird, muss entschieden werden, welche Modul-Instanz angeschlossen wird. Eine wichtige Voraussetzung ist jedoch, dass das Streckennetz von Modulinstanz m2 so konstruiert wurde, dass der Fahrer zu keinem Zeitpunkt Port2 und Port3 gleichzeitig sehen kann (s. Abbildung 7). Die Syntax zur Erstellung dieser Verknüpfungen finden Sie in dem Beispiel auf der vorherigen Seite. Dieses Vorgehen funktioniert auch mit Modulen, die mehr als drei Ausgänge haben.

Des Weiteren kann die dynamische Datenbasis von SILAB zur Erstellung bedingter Verknüpfungen von Szenario-Modulen genutzt werden. Das bedeutet, dass Sie den weiteren Modulablauf an definierte Bedingungen knüpfen können. Ein Beispiel wird in Abbildung 8 grafisch dargestellt. Der Fahrer soll die Modulinstanz m2, in der er mit einer kritischen Situation konfrontiert wird, erst erleben, wenn er das Assistenzsystem, z.B. ein Kollisionswarnsystem, am Ende der Modul-Instanz m1 aktiviert hat. Sollte dies nicht der Fall sein, soll er m1 wiederholt durchfahren.

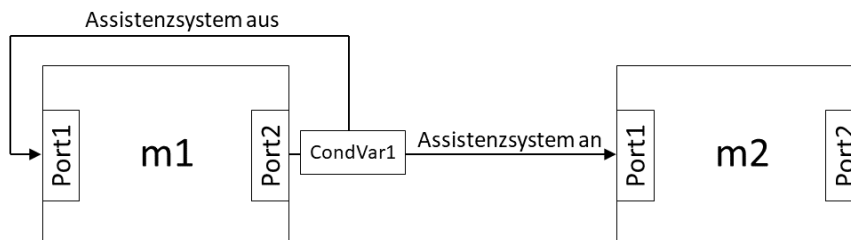


Abbildung 8. Visualisierung einer bedingten Verknüpfung von Szenario-Modulen.

Ein solcher bedingter Modulablauf würde folgendermaßen definiert werden:

```
### Verknüpfung der Instanzen zu einer Karte
Connections =
{
m1.Port2 -> CondVar1
(
= 1: m2.Port1,
= 0: m1.Port1
)
};
### /Verknüpfung der Instanzen zu einer Karte
```

Code 22. Beispiel für eine bedingte Verknüpfung von Szenario-modulen.

In dieser Definition ist CondVar1 ein Eingang der DPUSCNX. Dieser DPU-Eingang muss in der DPU-Konfiguration mit dem entsprechenden Ausgang der DPU des Assistenzsystems verknüpft werden. Im Beispiel liefert dieser Ausgang den Wert 1, wenn das Assistenzsystem an ist, und 0, wenn es nicht aktiviert wurde.

Die DPUSCNX hat 10 solcher Eingänge (CondVar1 bis CondVar10). Dementsprechend können Sie bedingte Verknüpfungen in Ihrem Modul-Ablauf einfügen, die von 10 unterschiedlichen Kriterien abhängen. Es können beliebige, in der Simulation vorliegende Daten verwendet werden, um einen bedingten Modul-Ablauf zu konstruieren. Die CondVar-Eingänge werden dabei immer mit Ausgängen von anderen DPUs verknüpft.

Bedingte Verknüpfungen können ebenfalls für die Erzeugung eines Streckennetzes aus Streckenstücken genutzt werden (Kapitel 6).

8.1 Festlegen der Aufsetzpunkte

Beim Start der Simulation wird der Fahrer auf dem Port einer Modul-Instanz oder einer Streckenstück-Instanz aufgesetzt. Die Stelle auf der Karte, an der sich der Fahrer zum Start der Simulation befindet, wird als Aufsetzpunkt (engl. „Set-up Point“) bezeichnet. Es besteht die Möglichkeit, mehrere Aufsetzpunkte für eine Karte zu definieren und somit vor jedem Simulationsstart auszuwählen, an welcher Stelle der Fahrer aufgesetzt werden soll.

Sie können in der Konfigurationsdatei definieren, zu Beginn welcher Module oder Streckenstücke (Course oder Area) sich die Aufsetzpunkte befinden sollen. Daraufhin wird Ihnen vor jedem Start einer Simulation ein sogenannter Launch-Wizard angezeigt, mit dem Sie den gewünschten Aufsetzpunkt durch Klicken auswählen können. Die Simulation wird daraufhin am Eingang des ausgewählten Streckenstücks gestartet. Abbildung 9 zeigt den Launch-Wizard, mit dem Sie in SILAB vor dem Start der Simulation den Aufsetzpunkt auswählen können.

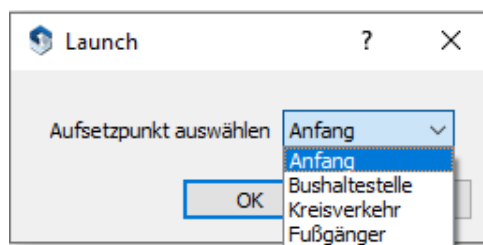


Abbildung 9. Launch-Wizard zur Auswahl des Aufsetzpunktes.

Die Aufsetzpunkte werden in der Szenariodefinition innerhalb des Abschnitts zur Erzeugung der Karte aus den Szenariomodulen festgelegt. Im Folgenden sehen Sie, wie die Definition von Aufsetzpunkten beispielsweise in der Konfigurationsdatei aussehen kann.

```

### Definition der Aufsetzpunkte
SetupPoints =
{
    ("Anfang", m1.course1.Begin, 1),
    ("Bushaltestelle", m2.area1.Port1),
    ("Kreisverkehr", m2.area2.Port2),
    ("Fußgänger", m3.Port1)
};
### /Definition der Aufsetzpunkte

```

Code 23. Beispiel für die Definition von Aufsetzpunkten.

Aufsetzpunkte werden als Elemente der Menge `SetupPoints = {...}`; aufgeführt. Jedes Element entspricht einem Aufsetzpunkt und hat eine dieser beiden Formen:

```

("<Name>", <Modul-Instanz>.<Streckenstück-Instanz>.<Port>)|
("<Name>", <Modul-Instanz>.<Streckenstück-Instanz>.<Port>, <Spurwechsel>)

```

"<Name>"	Name des Aufsetzpunktes, der im Auswahlmenü des Launch-Wizards angezeigt wird.
<Modulinstanz>	Name der Modulinstanz, auf der der Aufsetzpunkt liegen soll.
<Streckenstück-Instanz>	Name der Streckenstückinstanz (Course oder Area), auf der der Aufsetzpunkt liegen soll. Wenn sich der Aufsetzpunkt direkt auf einem Modulport befinden soll, entfällt der Eintrag <Streckenstückinstanz> (z.B. "Fußgänger", m3.Port1)
<Port>	Angabe des Ports eines Moduls oder einer Area, an dem der Aufsetzpunkt liegen soll (z.B. Port1). Bei einem Course erfolgt hier immer die Angabe ‚Begin‘ (z.B. "Anfang", m1.course1.Begin). Das Aufsetzen am Ende eines Course („End“) wird nicht unterstützt.
<Spurwechsel>	Normalerweise wird der Fahrer auf der Spur ganz rechts aufgesetzt (z.B. auf dem Standstreifen). Durch die zusätzliche Angabe von <Spurwechsel> kann angegeben werden, dass das Fahrzeug beim Aufsetzen um die hier angegebene Anzahl von Spuren nach links versetzt wird (z.B. "Anfang", m1.course1.Begin, 1)

Um den Aufsetzpunkt vor jedem Simulationsstart mithilfe des Launch-Wizards auswählen zu können, müssen zusätzlich zur Definition der Aufsetzpunkte in der Szenariodefinition die Launch-Daten innerhalb der Systemkonfiguration angepasst werden. Eine ausführliche Erklärung dazu finden Sie in Kapitel 8.6.2 im **HANDBUCH SILAB ESSENTIAL**. Zur Erstellung eines Launch-Wizards wie in Abbildung 9 müssten beispielsweise folgende Angaben in der Systemkonfiguration gemacht werden.

```

SILAB System
{

```

```
LaunchData1 = ("Aufsetzpunkt auswählen", SCN_SetupPoint,  
LD_COMBO_DEFAULT, "Anfang")  
};
```

Code 24. Beispiel-Konfiguration der Launch-Daten zur Auswahl der Aufsetzpunkte.

9 WIEDERVERWENDUNG VON SZENARIO-MODULEN

Sie haben die Möglichkeit, selbst erstellte Szenario-Module für verschiedene Fragestellungen wiederzuverwenden und flexibel mit neuen oder anderen bereits bestehenden Szenario-Modulen zu kombinieren.

Bitte beachten Sie beim Entwurf von wiederverwendbaren Szenarien grundsätzlich auf folgende Punkte:

- Der Sichtbereich des Fahrers sollte an den Ein- und Ausgängen des Szenario-Moduls eingeschränkt werden.
- Die Straßenquerschnitte der Szenario-Module sollten an den Ein- und Ausgängen kompatibel sein.
- Die in einem Szenario-Modul erzeugten Verkehrsteilnehmer sollten die Verkehrssituation im folgenden Modul nicht stören.

Es gibt zwei verschiedene Möglichkeiten zur Verwaltung bzw. Wiederverwendung von bereits bestehenden Szenarien. Zum einen können Sie das entsprechende Szenario-Modul als include-Datei auslagern und bei Wiederverwendung über die include-Funktion in die Szenarioefinition der Konfigurationsdatei einbinden. Das Prinzip ist analog zum Einbinden von Area-Modulen, für die eine Skriptdatei aus SILABAEEdit exportiert wurde (s. Kapitel 8).

Des Weiteren können Sie mit dem Programm SILABSessionEdit eigene Szenariendatenbanken erstellen und verwalten. **IM HANDBUCH SILAB ESSENTIAL** wird das Arbeiten mit SILABSessionEdit zur Erstellung von Sessions aus bereits bestehenden, mitgelieferten Szenarien-Datenbanken erklärt. Das gleiche Prinzip können Sie anwenden, um mit der übersichtlichen Benutzeroberfläche von SILABSessionEdit Ihre selbst erstellten Szenariomodule zu einer neuen Karte zusammenzufügen. In Kapitel 4 im Dokument **REFERENZ DATENBASIS SCNX** erfahren Sie, wie Sie eine neue Szenarien-Datenbank anlegen, neue Szenarien hinzufügen, bearbeiten und löschen können.

10 ERWEITERUNG DER GRAFISCHEN AUSGABE

Bei fast allen Elementen, die in einer Ansicht der SILAB-Visualisierung (SGEView) zu sehen sind, handelt es sich um grafische Objekte, z.B. die Landschaft, Straße, Fahrzeuge, Bäume und die Berge im Hintergrund. Der Lieferumfang von SILAB enthält bereits eine Vielzahl an grafischen Objekten, die für die Visualisierung der Simulation genutzt werden können, z.B. Straßenbeläge, Bebauungen, Verkehrsschilder, Häuser und Bäume. Eine große Bibliothek mit grafischen Objekten befindet sich in dem Ordner D:\SILAB\lib\graphics\default\models. Die mitgelieferten Objekte können mit dem Programm SGEObjectPrev, das sich im Lieferumfang von SILAB befindet, betrachtet werden.

Darüber hinaus besteht die Möglichkeit, eigene grafische Objekte zu entwerfen und in der Simulation einzusetzen. Eine Anleitung zum Entwurf grafischer Elementen finden Sie in Kapitel 3 (Erweiterung um grafische Objekte) im Dokument **REFERENZ SILAB VISUALISIERUNG SGE**. Dabei unterstützt Sie die Anwendung SGEObjectPrev, welche auch 3D-Modelle aus dem 3D-Austauschformat FBX importieren kann, das von den meisten gängigen 3D-Modellern unterstützt wird.

Es gibt drei Anwendungsfälle, für die Sie die selbst erstellten grafischen Objekte zur Erweiterung der grafischen Ausgabe nutzen können:

- Einfügen eines einzelnen, statischen grafischen Objekts in ein Szenario (z.B. Verkehrsschild, Baum, Haus): Wenn es sich um ein Szenario handelt, das mit SILABAEEdit erstellt wird, finden Sie die Vorgehensweise im Handbuch SILABAEEdit beschrieben. Bei Szenarien, die direkt über die Skriptsprache von SILAB entworfen werden, finden Sie die Beschreibung im Dokument Referenz Datenbasis SCNX.
- Bereitstellung neuer grafischer Objekte für SILABAEEdit (z.B. Häuserzeilen, Straßenbeläge oder Beläge für Verkehrsinseln und Straßenränder): Eine Anleitung finden Sie im Handbuch SILABAEEdit.
- Ein- und Ausblenden eines grafischen Objekts während der Simulation und/oder Veränderung seiner Position, Drehung und Skalierung: Für diesen Anwendungsfall wird die DPUSGEObject verwendet. Nähere Informationen finden Sie im Dokument DPUSGEObject.

11 GLOSSAR

Als **AREA** wird eine Art von Streckenstück bezeichnet, dessen Querschnittsprofil sich innerhalb des Streckenstücks ändert, z.B. eine Kreuzung, Autobahnauffahrt oder eine Spurverengung.

AUFSETZPUNKT ist die Stelle in einer Karte, an der die Simulation gestartet werden kann. Aufsetzpunkte werden zuvor definiert und können vor jedem Start der Simulation mit dem Launch-Wizard ausgewählt werden.

COURSE ist ein beliebig langer Abschnitt einer Straße, der keine Abzweigungen hat und bei dem sich das Querschnittsprofil (z.B. Anzahl der Spuren, Spurbreiten, Belag der Spuren) nicht ändert. Ein Course stellt eine Art von Streckenstück dar.

DPU (Data Processing Units) sind grundlegende funktionale Softwaremodule einer Simulation mit SILAB, die während der Simulation verschiedene Aufgaben erfüllen.

FLOWPOINT definieren in der Verkehrssimulation, wann und wie einzelne Verkehrsteilnehmer oder ganze Verkehrsflüsse aktiviert und deaktiviert werden.

FUßGÄNGERSIMULATION MOV ermöglicht die Simulation von Fußgängern innerhalb von Stadt-szenarien, d.h. Streckenstücke des Typs Area. Sie gibt vor, wie sich die Fußgänger auf einem Streckennetz verhalten sollen.

GRAFISCHE OBJEKTE sind einzelne Objekte, wie z.B. Verkehrsschilder, Straßenpfosten, Fahrbahnmarkierungen in Verkehrsknoten, Bäume und Häuser, die in der simulierten Landschaft von Streckenstücken platziert werden können.

HÖHENPROFIL eines Streckenstücks, das aus einer Abfolge von Segmenten besteht, die entweder eben sind oder eine Steigung bzw. ein Gefälle haben.

INSTANZIIERUNG beschreibt das Erzeugen eines Objekts (DPU-Instanz) aus einer bestimmten Klasse (DPU). Aus einer DPU können verschiedene DPU-Instanzen abgeleitet werden.

Als **KARTE** (engl. **MAP**) wird in SILAB ein gesamtes Streckennetzwerk bezeichnet, auf dem man in der Simulation fahren kann.

LAGEPLAN bezeichnet den Straßenverlauf eines Streckenstücks in der Ebene, welcher sich aus Geraden einer definierten Länge und Kurven einer definierten Länge und Radius zusammensetzt.

LAUNCH-DATEN sind Informationen, die Sie vor jedem Start einer Simulation den DPU-Instanzen mittels Eingaben in einem Dialogfenster mitteilen können, z.B. Informationen zur Datenaufzeichnung, Videoaufzeichnung oder Auswahl des Aufsetzpunktes.

LAUNCH-WIZARD bezeichnet das Dialog-Fenster, mit dem vor dem Simulationsstart die Launch-Daten abgefragt werden.

QUERSCHNITTSPROFIL definiert den Aufbau einer Straße im Querschnitt und enthält Informationen über die Anzahl der Spuren, Breite und Höhe der Spuren, Fahrtrichtung und Befahrbarkeit, Straßenbelag, Linienmarkierungen zwischen den Spuren, Bebauungen zwischen bzw. neben den Spuren (z.B. Leuchtpfosten, Leitplanken), Bebauungen auf den Spuren (z.B. Hecke auf einem Begrenzungstreifen in der Mitte einer Autobahn) und zu den Seitenstreifen.

SCN/SCNX steht für ‚Scenery‘ ist die Datenbasis für die SILAB-Fahrsimulation, die Informationen über den Verlauf, die Gestaltung und den Aufbau des befahrbaren Streckennetzes, sowie das Aussehen und Höhenprofil der Umgebung enthält. SCNX ist die weiterentwickelte Version von SCN (X steht für Extended).

SCNXSGE ist das Programmmodul der SILAB-Visualisierung, das für eine realistische Darstellung des simulierten Szenarios auf den Anzeigegeräten des Fahrsimulators (Monitore oder Projektoren) sorgt. SGE steht für ‚Simulator Graphics Enhanced‘.

SILABADMIN ist ein Programm von SILAB, mit dem die an der Simulation beteiligten Rechner gesteuert und überwacht werden.

SILABAEDIT ist ein Programm des SILAB-Softwarepakets, das als grafischer Editor zur Gestaltung von Areas benutzt wird.

SILABSESSIONEDIT ist ein Programm des SILAB-Softwarepakets, mit dem Sessions aus mitgelieferten Szenarien-Datenbanken zusammengestellt werden können.

SESSION bezeichnet eine Simulationsstrecke in SILAB, die aus einer Abfolge von verschiedenen Szenarien besteht.

STRECKEN-EVENTS werden in SILAB eingesetzt, um automatisch bestimmte Ereignisse auszulösen. Es handelt sich dabei um unsichtbare Markierungen auf den Spuren der Strecke, die durch Überfahren des EGO-Fahrzeugs oder eines anderen Verkehrsteilnehmers aktiviert werden und somit das definierte Ereignis auslösen.

STRECKENNETZ ist das Ergebnis der Verknüpfung mehrerer Streckenstücke.

STRECKENSTÜCK ist der kleinste Bestandteil einer Karte. Streckenstücke können bspw. ein durchgängiger Abschnitt einer Straße, eine Kreuzung, eine Autobahnauffahrt oder -abfahrt.

SZENARIEN-DATENBANK bezeichnet die Sammlung von Szenarien in unterschiedlichen Verkehrsumgebungen, aus denen mit dem Programm SILABSessionEdit nach einem Baukasten-Prinzip eine Session erstellt werden kann.

Als **SZENARIO-MODUL** wird eine abgeschlossene Einheit aus einem Streckennetz und einer Verkehrsdefinition bezeichnet. Diese Szenario-Module werden zu einer bestimmten Reihenfolge verknüpft, in der sie dem Fahrer letztlich während der Simulation dargeboten werden (Modul-Abfolge).

TRF/TRFX steht für ‚Traffic‘ und ist ein Verbund aus verschiedenen DPU's, die in SILAB für die Simulation anderer Verkehrsteilnehmer zuständig sind. TRFX ist die weiterentwickelte Version von TRF (X steht für Extended).

UMGEBUNGSLANDSCHAFT bezeichnet die Landschaft, die ein Streckenstück umgibt. Diese wird in SILAB durch Landschaftsgeneratoren modelliert.

VERKEHRSDEFINITION beinhaltet Vorschriften, wie sich andere einzelne Verkehrsteilnehmer oder ganze Verkehrsflüsse auf einem vorgegebenen Streckennetz verhalten sollen.

VERHALTENSMODELL (engl. behaviour machine) steuert das Verhalten von anderen Verkehrsteilnehmern in der Simulation. Verkehrsteilnehmern können verschiedene Verhaltensmodelle in einer festen Hierarchie zugeordnet werden.

VERHALTENSSCHEMA (engl. behaviour scheme) beschreibt eine vordefinierte Hierarchie aus verschiedenen Verhaltensmodellen, die das Verhalten von anderen Verkehrsteilnehmern in der Simulation steuern.